Programming with Python

Course code 202044504

Assignment 1 — Solutions

B.Tech. Information Technology · Semester V

Student name	Pearl Parmar
Enrollment no.	12402080601104
Batch	2B8
Date of submission	

10 programs · Python 3.10+ · standard library only · 66 of 66 tests passed

Contents

Q1. Campus Merit Analyzer using Compound Data Structures . . . 3

Q2. Optimized Password Audit with Pattern Constraints . . . 7

Q3. Recursive Expression Engine with Memoization . . . 11

Q4. Exception-Safe CSV Transaction Splitter . . . 16

Q5. Object-Oriented Bank Settlement System . . . 20

Q6. Python Module Dependency Resolver . . . 26

Q7. Interactive Formula Validator with Custom Exceptions . . . 30

Q8. Compressed Log Index using Pickle and Zip . . . 34

Q9. Threaded Job Scheduler Simulation . . . 39

Q10. Tkinter Assignment Tracker with File Persistence . . . 43

How the answers are organised

- Each question has: the problem in brief, the approach and data structures, assumptions where the paper is silent, time and space complexity, how to run it, the complete source code, and the test evidence.
- Every program validates its input, uses meaningful function names and comments, and was run on the sample test, boundary tests, invalid-input tests, at least three self-made tests and a large test at the maximum constraints.
- Source files follow the convention `Enrollment_AssignmentNo_QNo.py` with `ENROLLMENT` as a placeholder for the enrollment number.
- Environment: Python 3.12 on Linux, standard library only (Tkinter for Q10). The same files run unchanged on Windows and macOS with Python 3.10 or newer.

Test summary

Question	Tests	Passed	Largest input tested
Q1. Campus Merit Analyzer using Compound Data Structures	8	8	n = 100,000 students, m = 12
Q2. Optimized Password Audit with Pattern Constraints	6	6	b = 10,000, n = 100,000
Q3. Recursive Expression Engine with Memoization	8	8	200,000-variable chain
Q4. Exception-Safe CSV Transaction Splitter	6	6	1,000,000 CSV rows
Q5. Object-Oriented Bank Settlement System	7	7	100,000 accounts, ~300,000 ops
Q6. Python Module Dependency Resolver	7	7	n = 200,000, e = 500,000
Q7. Interactive Formula Validator with Custom Exceptions	5	5	100,000 input lines
Q8. Compressed Log Index using Pickle and Zip	7	7	2,000,000 log lines
Q9. Threaded Job Scheduler Simulation	6	6	w = 64, n = 200,000
Q10. Tkinter Assignment Tracker with File Persistence	6	6	5,000 records in GUI table

Q1. Campus Merit Analyzer using Compound Data Structures

CO-2, CO-4 | Bloom L4 Analyze | Lists, tuples, dictionaries, sorting, hashing, aggregation

Problem

Store student records (enrollment, name, semester, CPI, m subject marks) using lists, tuples and dictionaries. For every semester print the top K students by CPI (tie: higher average marks, then lexicographically smaller enrollment) and print the topper(s) of every subject.

Approach

- Each student is stored as one **tuple** (enrollment, name, semester, cpi, total, marks); all tuples are kept in a **list**. Tuples are immutable, so a record cannot be changed by mistake after validation.
- CPI is converted to an integer number of hundredths (8.90 becomes 890). Comparing integers means 8.9 and 8.90 are exactly equal, with no floating-point error.
- A **dictionary** (defaultdict) maps semester to the list of its students: grouping is a single O(n) hash-based pass. Each group is sorted once with the key (-cpi, -total, enrollment), which encodes all three ranking rules.
- The total of marks is compared instead of the average: every student has the same m subjects, so the order is identical and no division is needed.
- Subject toppers are found in one pass: for each subject the best mark and the list of enrollments holding it are kept; ties are all printed, in sorted order.
- Validation: header shape, field count per line, ranges of n, k, m, semester, CPI and marks, and duplicate enrollments. Every problem raises `InputError` with the line number.

Complexity

Time O(n·m + n log n): reading all marks is n·m; the per-semester sorts total at most n log n. **Space** O(n·m) for the records.

Note on the sample output. The question paper prints `Semester 3: 2205 2203` and says Esha has the higher average. Her marks 91+90+88 total 269 (average 89.67), while Chaitra's 91+88+92 total 271 (average 90.33). By the paper's own rule 2203 must come first, so this program prints `Semester 3: 2203 2205`. All other sample lines match exactly.

How to run

```
python ENROLLMENT_1_Q1.py < sample_input.txt
```

Source code — ENROLLMENT_1_Q1.py

```python
"""
Q1. Campus Merit Analyzer using Compound Data Structures
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-2, CO-4

Reads student records, prints the top K students of every semester and the
topper(s) of every subject.

Data structures used
    * tuple  -> one immutable student record (enrollment, name, sem, cpi, avg, marks)
    * list   -> all records of one semester, marks of one student
    * dict   -> semester -> list of records   (hash based grouping)
                subject index -> (best mark, list of enrollments)

Ranking rule inside a semester
    1. higher CPI first
    2. if CPI equal  -> higher average marks first
    3. if still tied -> lexicographically smaller enrollment number first

Run     : python ENROLLMENT_1_Q1.py < input.txt
Time    : O(n*m + n log n)      Space : O(n*m)
"""

import sys
from collections import defaultdict


class InputError(Exception):
    """Raised when the input does not follow the required format/constraints."""


def parse_int(token, name, low, high):
    """Convert token to int and check low <= value <= high."""
    try:
        value = int(token)
    except ValueError:
        raise InputError(f"{name} must be an integer, got '{token}'") from None
    if not low <= value <= high:
        raise InputError(f"{name}={value} is outside [{low}, {high}]")
    return value


def parse_cpi(token):
    """CPI is kept as an integer number of hundredths (8.90 -> 890).

    Comparing integers avoids floating point surprises such as
    8.9 != 8.90000001 when two CPIs are meant to be equal.
    """
    try:
        value = round(float(token) * 100)
    except ValueError:
        raise InputError(f"CPI must be a number, got '{token}'") from None
    if not 0 <= value <= 1000:
        raise InputError(f"CPI {token} is outside [0.0, 10.0]")
    return value


def read_records(lines):
    """Parse the whole input. Returns (k, m, list_of_record_tuples)."""
    if not lines:
        raise InputError("empty input")
    header = lines[0].split()
    if len(header) != 3:
        raise InputError("first line must be: n k m")
    n = parse_int(header[0], "n", 1, 100000)
    k = parse_int(header[1], "k", 1, 50)
    m = parse_int(header[2], "m", 1, 12)
    if len(lines) - 1 < n:
        raise InputError(f"expected {n} student lines, found {len(lines) - 1}")

    records = []
    seen = set()                        # detect duplicate enrollment numbers
```

```python
    for line_no in range(1, n + 1):
        parts = lines[line_no].split()
        if len(parts) != 4 + m:
            raise InputError(f"line {line_no + 1}: expected {4 + m} fields, got {len(parts)}")
        enrollment, name = parts[0], parts[1]
        if enrollment in seen:
            raise InputError(f"line {line_no + 1}: duplicate enrollment {enrollment}")
        seen.add(enrollment)
        semester = parse_int(parts[2], "semester", 1, 8)
        cpi = parse_cpi(parts[3])
        marks = tuple(parse_int(t, "mark", 0, 100) for t in parts[4:])
        total = sum(marks)            # average compared as total (m is the same for all)
        records.append((enrollment, name, semester, cpi, total, marks))
    return k, m, records


def semester_toppers(records, k):
    """Group by semester (dict) and sort each group once.

    Sorting key (-cpi, -total, enrollment) puts the best student first.
    Returns a dict: semester -> list of top-k enrollments.
    """
    groups = defaultdict(list)
    for rec in records:
        groups[rec[2]].append(rec)

    result = {}
    for sem in sorted(groups):
        ranked = sorted(groups[sem], key=lambda r: (-r[3], -r[4], r[0]))
        result[sem] = [r[0] for r in ranked[:k]]
    return result


def subject_toppers(records, m):
    """One pass over all marks. For each subject keep the best mark and
    every enrollment that achieved it (ties are all printed)."""
    best = [-1] * m
    holders = [[] for _ in range(m)]
    for enrollment, _, _, _, _, marks in records:
        for j, mark in enumerate(marks):
            if mark > best[j]:
                best[j] = mark
                holders[j] = [enrollment]
            elif mark == best[j]:
                holders[j].append(enrollment)
    return {f"S{j + 1}": sorted(holders[j]) for j in range(m)}


def solve(text):
    """Return the complete output as a string (used by main and by tests)."""
    lines = [ln for ln in text.splitlines() if ln.strip()]
    k, m, records = read_records(lines)
    out = []
    for sem, top in semester_toppers(records, k).items():
        out.append(f"Semester {sem}: " + " ".join(top))
    for subject, top in subject_toppers(records, m).items():
        out.append(f"{subject}: " + " ".join(top))
    return "\n".join(out)


def main():
    try:
        print(solve(sys.stdin.read()))
    except InputError as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 8 of 8 passed

#	Test case	Input	Output (expected = actual)	Result

1	**Sample test**
	sample
		`5 2 3`
		`2201 Asha 5 8.90 90 82 75`
		`2202 Bharat 5 8.90 88 95 70`
		`2203 Chaitra 3 9.10 91 88 92`
		`2204 Dev 5 7.80 76 80 85`
		`2205 Esha 3 9.10 91 90 88`
			`Semester 3: 2203 2205`
			`Semester 5: 2202 2201`
			`S1: 2203 2205`
			`S2: 2202`
			`S3: 2203`
				PASS

2	**Boundary: n=1, k=1, m=1, CPI 10.0, marks 100** boundary	`1 1 1` `9999 Zed 8 10.0 100`	`Semester 8: 9999` `S1: 9999`	**PASS**

3	**k larger than group; CPI and average tie -> smaller enrollment**
	custom
		`3 5 2`
		`100 A 1 8.5 50 60`
		`101 B 1 8.5 60 50`
		`099 C 2 7.0 0 0`
			`Semester 1: 100 101`
			`Semester 2: 099`
			`S1: 101`
			`S2: 100`
				PASS

4	**CPI 8.9 and 8.90 treated as equal (no float error)**
	custom
		`2 1 1`
		`5 X 3 8.9 40`
		`4 Y 3 8.90 40`
			`Semester 3: 4`
			`S1: 4 5`
				PASS

5	**All marks zero -> every student tops**
	custom
		`3 1 1`
		`30 P 2 6.0 0`
		`10 Q 2 6.0 0`
		`20 R 2 6.0 0`
			`Semester 2: 10`
			`S1: 10 20 30`
				PASS

6	**Invalid: mark above 100** invalid	`1 1 2` `1 A 1 9.0 50 101`	`INPUT ERROR: mark=101 is outside [0, 1 00]`	**PASS**
7	**Invalid: wrong number of marks** invalid	`1 1 3` `1 A 1 9.0 50 60`	`INPUT ERROR: line 2: expected 7 fields , got 6`	**PASS**
8	**Max size: n=100000, k=50, m=12** performance	`100000 random students, 12 subjects`	`finished in 0.80 s`	**PASS**

Sample test: The paper prints 'Semester 3: 2205 2203', but by its own rule 2203 (avg 90.33) beats 2205 (avg 89.67); see the note in the answer.

Q2. Optimized Password Audit with Pattern Constraints

CO-1, CO-3 | Bloom L5 Evaluate | Strings, regular expressions, trie, validation

Problem

Classify n passwords as STRONG, WEAK_LENGTH, WEAK_PATTERN or COMPROMISED. Strong means: length 6 to 12, at least one lower-case, upper-case, digit and one of $#@, no banned word as a case-insensitive substring, and no character repeated more than 3 times in a row.

Approach

- All banned words are inserted (lower-cased) into a **trie**. A breadth-first pass adds **failure links**, turning the trie into an **Aho-Corasick automaton**. Each password is then scanned once, left to right, and the automaton reports whether any banned word occurs; the number of banned words no longer matters.
- Each state stores a flag 'some banned word ends here'. The flag is inherited through failure links, so a short word such as `bc` is found even while matching the longer word `abcd` (covered by a test).
- Regular expressions implement the pattern rules: `[a-z]`, `[A-Z]`, `\d`, `[$#@]` and `(.)\1{3}`, which matches the same character 4 or more times in a row.
- Checks run in the order length, banned word, pattern, and the first failing rule decides the class. This order reproduces the sample (password 2 is COMPROMISED, not WEAK_PATTERN).
- Passwords are read with only the line ending removed, so spaces inside a password are kept and checked.

Complexity

Time O(total banned length + total password length): building the automaton is linear in the banned text and every password character is processed in amortised O(1). The O(n·b·L) brute force is avoided. **Space** O(total banned length) for the automaton.

Assumptions: a password missing a character class is WEAK_PATTERN; a password longer than 100 characters is reported as WEAK_LENGTH.

How to run

```
python ENROLLMENT_1_Q2.py < sample_input.txt
```

Source code — ENROLLMENT_1_Q2.py

```python
"""
Q2. Optimized Password Audit with Pattern Constraints
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-3

Each password is classified as
    WEAK_LENGTH  - length not in [6, 12]
    COMPROMISED  - contains a banned word (case-insensitive substring)
    WEAK_PATTERN - missing lower/upper/digit/special($#@) OR has a character
                   repeated more than 3 times consecutively
    STRONG       - passes every rule

Check order (assumption, consistent with the sample): length -> banned word ->
pattern. A password gets the FIRST failing class.

Banned-word search uses the Aho-Corasick automaton (a trie + failure links),
so all banned words are searched in ONE left-to-right scan of the password.
Regular expressions are used for the character-class and repetition rules.

Run     : python ENROLLMENT_1_Q2.py < input.txt
Time    : O(total banned length + total password length)
Space   : O(total banned length)
"""

import re
import sys
from collections import deque


class InputError(Exception):
    """Raised for malformed input."""


# ---------- regular expressions for the pattern rules ----------
HAS_LOWER = re.compile(r"[a-z]")
HAS_UPPER = re.compile(r"[A-Z]")
HAS_DIGIT = re.compile(r"\d")
HAS_SPECIAL = re.compile(r"[$#@]")
REPEAT_4 = re.compile(r"(.)\1{3}")          # same char 4+ times in a row


class AhoCorasick:
    """Multi-pattern substring matcher.

    goto[state]  : dict char -> next state   (the trie)
    fail[state]  : longest proper suffix that is also a trie prefix
    term[state]  : True if some banned word ends at this state
                   (including words ending at a suffix via fail links)
    """

    def __init__(self, words):
        self.goto = [{}]
        self.fail = [0]
        self.term = [False]
        for word in words:
            self._insert(word.lower())
        self._build_links()

    def _insert(self, word):
        state = 0
        for ch in word:
            nxt = self.goto[state].get(ch)
            if nxt is None:
                nxt = len(self.goto)
                self.goto[state][ch] = nxt
                self.goto.append({})
                self.fail.append(0)
                self.term.append(False)
            state = nxt
        self.term[state] = True

    def _build_links(self):
```

```
        """BFS over the trie to compute failure links."""
        queue = deque(self.goto[0].values())       # depth-1 nodes fail to root
        while queue:
            state = queue.popleft()
            for ch, child in self.goto[state].items():
                queue.append(child)
                f = self.fail[state]
                while f and ch not in self.goto[f]:
                    f = self.fail[f]
                self.fail[child] = self.goto[f].get(ch, 0)
                if self.fail[child] == child:      # safety for root children
                    self.fail[child] = 0
                # inherit "a word ends here" from the suffix state
                self.term[child] = self.term[child] or self.term[self.fail[child]]

    def contains_any(self, text):
        """True if any banned word occurs in text (text already lower-case)."""
        goto, fail, term = self.goto, self.fail, self.term
        state = 0
        for ch in text:
            while state and ch not in goto[state]:
                state = fail[state]
            state = goto[state].get(ch, 0)
            if term[state]:
                return True
        return False


def classify(password, matcher):
    """Return the classification string for a single password."""
    if not 6 <= len(password) <= 12:
        return "WEAK_LENGTH"
    if matcher.contains_any(password.lower()):
        return "COMPROMISED"
    if not (HAS_LOWER.search(password) and HAS_UPPER.search(password)
            and HAS_DIGIT.search(password) and HAS_SPECIAL.search(password)):
        return "WEAK_PATTERN"
    if REPEAT_4.search(password):
        return "WEAK_PATTERN"
    return "STRONG"


def read_count(lines, idx, name, low, high):
    if idx >= len(lines):
        raise InputError(f"missing value for {name}")
    try:
        value = int(lines[idx].strip())
    except ValueError:
        raise InputError(f"{name} must be an integer, got '{lines[idx].strip()}'") from None
    if not low <= value <= high:
        raise InputError(f"{name}={value} outside [{low}, {high}]")
    return value


def solve(text):
    # keep spaces inside passwords; only strip the line ending
    lines = text.split("\n")
    lines = [ln.rstrip("\r") for ln in lines]
    b = read_count(lines, 0, "b", 1, 10000)
    banned = [w.strip() for w in lines[1:1 + b]]
    if len(banned) < b or any(not w for w in banned):
        raise InputError("banned word list is incomplete or has empty words")
    if sum(map(len, banned)) > 200000:
        raise InputError("total banned characters exceed 200000")
    n = read_count(lines, 1 + b, "n", 1, 100000)
    passwords = lines[2 + b: 2 + b + n]
    if len(passwords) < n:
        raise InputError(f"expected {n} passwords, found {len(passwords)}")

    matcher = AhoCorasick(banned)
    out = []
    for i, pwd in enumerate(passwords, start=1):
```

```
        if len(pwd) > 100:
            out.append(f"{i}: WEAK_LENGTH")
            continue
        out.append(f"{i}: {classify(pwd, matcher)}")
    return "\n".join(out)


def main():
    try:
        print(solve(sys.stdin.read()))
    except InputError as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 6 of 6 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	2 admin college 5 Abc12@ adminA1@ AAAA1@b short Good9#	1: STRONG 2: COMPROMISED 3: WEAK_PATTERN 4: WEAK_LENGTH 5: STRONG	PASS
2	**Boundary lengths 6, 12 and 13** boundary	1 zzz 3 Ab1@xy Ab1@xyzwvuts Ab1@xyzwvutsr	1: STRONG 2: STRONG 3: WEAK_LENGTH	PASS
3	**Aho-Corasick: suffix match, case-insensitive, missing digit** custom	4 abcd bc ADMIN she 5 Xabc1@yz uSHEr1@A xHiS9#ab AdMiN99$x NoDigit@@	1: COMPROMISED 2: COMPROMISED 3: STRONG 4: COMPROMISED 5: WEAK_PATTERN	PASS
4	**Repetition: exactly 3 allowed, 4 rejected; '!' is not a valid special** custom	1 qq 5 aaa1@B aaaa1@B 1111aA$ @@@#Aa1 Aa1!xyz	1: STRONG 2: WEAK_PATTERN 3: WEAK_PATTERN 4: STRONG 5: WEAK_PATTERN	PASS
5	**Invalid: banned count not a number** invalid	x admin 1 Abc12@	INPUT ERROR: b must be an integer, got 'x'	PASS
6	**Max size: b=10000 (200000 chars), n=100000** performance	10000 banned words x 20 chars, 100000 passwords	finished in 0.71 s	PASS

Q3. Recursive Expression Engine with Memoization

CO-1, CO-2 | Bloom L6 Create | Recursion, stacks, dictionaries, parsing, memoization, cycle detection

Problem

Evaluate an integer expression with +, -, *, parentheses and variables, where each variable is defined by another expression. Print the value, CYCLE for cyclic dependencies, or INVALID for bad syntax. Repeated variable evaluations must be memoised.

Approach

- A single regular expression tokenises the text into numbers, names and operators.
- A **recursive-descent parser** (one method per grammar rule: expr, term, factor) builds a small tree of tuples, so * binds tighter than + and -. Chains such as 1+1+...+1 are stored flat, so long inputs do not create deep trees. The parser also records which variables each expression uses.
- Variables are resolved with a **depth-first search** over the dependency graph using an explicit stack, then evaluated in post-order, so every dependency is already known. Values go into a **memo dictionary**, so each definition is evaluated exactly once even if it is used thousands of times.
- **Cycle detection** uses DFS colouring: a variable is IN_PROGRESS while its dependencies are being resolved and DONE afterwards. Reaching an IN_PROGRESS variable again is a back edge, which means a cycle.
- The explicit stack matters: the first version used plain recursion and crashed with RecursionError on a 200,000-variable chain. The test for that chain is kept in the suite.

Complexity

Time O(total expression length): tokenising, parsing and evaluating each touch every token once, and memoisation ensures each variable is evaluated once. **Space** O(total expression length) for the trees, memo table and stack.

Assumptions: unary minus (e.g. `- -1`) is accepted; an undefined variable, a duplicate definition or a syntax error in any definition gives INVALID; a cycle is reported when it is reachable from the expression being evaluated.

How to run

```
python ENROLLMENT_1_Q3.py < sample_input.txt
```

Source code — ENROLLMENT_1_Q3.py

```python
"""
Q3. Recursive Expression Engine with Memoization
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-2

Grammar (recursive descent, one function per rule):
    expr   := term   (('+' | '-') term)*
    term   := factor ('*' factor)*
    factor := NUMBER | NAME | '(' expr ')' | '-' factor     (unary minus allowed)

Steps
    1. Tokenise every definition and the final expression with one regex.
    2. Parse each into a small tree (tuples). '+' / '-' chains and '*' chains
       are stored FLAT (a list of operands) so long inputs such as
       1+1+1+...+1 do not create deep trees.
    3. Before the final expression is evaluated, every variable it needs is
       resolved by a depth-first search over the dependency graph. Each value
       is memoised in a dict, so each definition is evaluated at most once.
       The DFS uses an explicit stack (not Python recursion), so a chain of
       200000 variables cannot overflow the call stack.
    4. Cycle detection uses three states (DFS colouring):
       unvisited / IN_PROGRESS / DONE. Meeting an IN_PROGRESS variable again
       means a cyclic dependency -> CYCLE.
    5. Each single expression tree is evaluated recursively (recursion depth
       = bracket nesting only).

Output : integer value, CYCLE, or INVALID (syntax error, undefined variable,
         duplicate definition, malformed input).
Assumption: a cycle is reported only if it is reachable from the expression
            being evaluated (unused cyclic definitions do not affect the answer).

Run     : python ENROLLMENT_1_Q3.py < input.txt
Time    : O(total expression length)   Space : O(total expression length)
"""

import re
import sys
import threading

TOKEN_RE = re.compile(r"\s*(?:(\d+)|([A-Za-z_]\w*)|(\S))")
NAME_RE = re.compile(r"[A-Za-z_]\w*\Z")


class InvalidExpression(Exception):
    """Syntax error / undefined variable / bad input."""


class CyclicDependency(Exception):
    """A variable depends on itself (directly or indirectly)."""


def tokenize(text):
    """Return list of (kind, value) tokens: kind is 'num', 'name' or 'op'."""
    tokens = []
    pos = 0
    text = text.rstrip()
    while pos < len(text):
        m = TOKEN_RE.match(text, pos)
        if not m:            # only trailing spaces could fail; treat as end
            break
        num, name, op = m.groups()
        if num is not None:
            tokens.append(("num", int(num)))
        elif name is not None:
            tokens.append(("name", name))
        else:
            if op not in "+-*()":
                raise InvalidExpression(f"unexpected character '{op}'")
            tokens.append(("op", op))
        pos = m.end()
    return tokens

```

```python

class Parser:
    """Recursive-descent parser producing a tuple tree."""

    def __init__(self, tokens):
        self.tokens = tokens
        self.i = 0
        self.names = []          # variables used by this expression (dependencies)

    def peek(self):
        return self.tokens[self.i] if self.i < len(self.tokens) else (None, None)

    def take(self):
        tok = self.peek()
        self.i += 1
        return tok

    def parse(self):
        if not self.tokens:
            raise InvalidExpression("empty expression")
        tree = self.expr()
        if self.i != len(self.tokens):
            raise InvalidExpression(f"unexpected token {self.peek()[1]!r}")
        return tree

    def expr(self):
        parts = [(1, self.term())]               # (sign, subtree)
        while self.peek() in (("op", "+"), ("op", "-")):
            sign = 1 if self.take()[1] == "+" else -1
            parts.append((sign, self.term()))
        return parts[0][1] if len(parts) == 1 else ("sum", parts)

    def term(self):
        factors = [self.factor()]
        while self.peek() == ("op", "*"):
            self.take()
            factors.append(self.factor())
        return factors[0] if len(factors) == 1 else ("prod", factors)

    def factor(self):
        kind, value = self.take()
        if kind == "num":
            return ("num", value)
        if kind == "name":
            self.names.append(value)
            return ("var", value)
        if (kind, value) == ("op", "-"):
            return ("neg", self.factor())
        if (kind, value) == ("op", "("):
            inner = self.expr()
            if self.take() != ("op", ")"):
                raise InvalidExpression("missing ')'")
            return inner
        raise InvalidExpression("operand expected")


class Engine:
    """Definitions, memo table and DFS colours.

    Variable dependencies are resolved with an ITERATIVE depth-first search
    (explicit stack), so a chain of 200000 variables needs no deep recursion.
    Variables are evaluated in post-order: when a variable is evaluated, all
    variables it uses are already in the memo table.
    """

    IN_PROGRESS, DONE = 1, 2

    def __init__(self, definitions):
        self.defs = definitions          # name -> (tree, list of used names)
        self.memo = {}                   # name -> int value (memoisation)
        self.state = {}                  # name -> IN_PROGRESS / DONE

```

```
    def resolve(self, start):
        """Evaluate `start` and everything it depends on, deepest first."""
        if self.state.get(start) == self.DONE:
            return
        if start not in self.defs:
            raise InvalidExpression(f"undefined variable '{start}'")
        self.state[start] = self.IN_PROGRESS
        stack = [(start, iter(self.defs[start][1]))]
        while stack:
            name, deps = stack[-1]
            for dep in deps:
                st = self.state.get(dep)
                if st == self.DONE:
                    continue                            # already memoised
                if st == self.IN_PROGRESS:
                    raise CyclicDependency(dep)         # back edge -> cycle
                if dep not in self.defs:
                    raise InvalidExpression(f"undefined variable '{dep}'")
                self.state[dep] = self.IN_PROGRESS
                stack.append((dep, iter(self.defs[dep][1])))
                break
            else:                                       # all deps done
                stack.pop()
                self.memo[name] = self.evaluate(self.defs[name][0])
                self.state[name] = self.DONE

    def evaluate(self, node):
        """Evaluate one tree. Every variable in it is already memoised."""
        kind = node[0]
        if kind == "num":
            return node[1]
        if kind == "var":
            return self.memo[node[1]]
        if kind == "neg":
            return -self.evaluate(node[1])
        if kind == "sum":
            total = 0
            for sign, sub in node[1]:
                total += sign * self.evaluate(sub)
            return total
        if kind == "prod":
            result = 1
            for sub in node[1]:
                result *= self.evaluate(sub)
            return result
        raise InvalidExpression("corrupt tree")      # should never happen


def parse_expression(text):
    parser = Parser(tokenize(text))
    return parser.parse(), parser.names


def solve(text):
    lines = [ln for ln in text.splitlines() if ln.strip()]
    try:
        if not lines:
            raise InvalidExpression("empty input")
        try:
            v = int(lines[0])
        except ValueError:
            raise InvalidExpression("first line must be v") from None
        if not 1 <= v <= 200000 or len(lines) < v + 2:
            raise InvalidExpression("wrong number of lines")

        definitions = {}
        for line in lines[1:v + 1]:
            if "=" not in line:
                raise InvalidExpression("definition without '='")
            name, expr = line.split("=", 1)
            name = name.strip()
            if not NAME_RE.match(name) or name in definitions:
```

```
                raise InvalidExpression(f"bad or duplicate variable name '{name}'")
            definitions[name] = parse_expression(expr)

        target, used = parse_expression(lines[v + 1])
        engine = Engine(definitions)
        for name in used:
            engine.resolve(name)
        return str(engine.evaluate(target))
    except CyclicDependency:
        return "CYCLE"
    except InvalidExpression:
        return "INVALID"


def main():
    print(solve(sys.stdin.read()))


if __name__ == "__main__":
    # Only deeply nested brackets ((((...)))) need recursion now, so give the
    # parser a large stack and recursion limit for that case.
    sys.setrecursionlimit(1_000_000)
    threading.stack_size(512 * 1024 * 1024)
    worker = threading.Thread(target=main)
    worker.start()
    worker.join()
```

Test cases — 8 of 8 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	3 a = 2 + 3 b = a * 4 c = b - (a + 1) c + 10	24	**PASS**
2	**Cycle a -> b -> c -> a** custom	3 a = b + 1 b = c * 2 c = a - 3 a + 1	CYCLE	**PASS**
3	**Self reference a = a** boundary	1 a = a a	CYCLE	**PASS**
4	**Missing ')'** invalid	1 x = (2 + 3 x	INVALID	**PASS**
5	**Undefined variable** invalid	1 x = y + 1 x	INVALID	**PASS**
6	**Precedence, nested brackets, unary minus** custom	2 p = 2 * (3 + 4) - 5 q = p * p - -1 q - 2 * 3	76	**PASS**
7	**64-bit boundary value** boundary	1 big = 9223372036854775807 big - 1	9223372036854775806	**PASS**
8	**Memoization: 200000-deep chain, each var used 3 times** performance	x(i) = x(i-1) + x(i-1) - x(i-1); witho ut memo this is 3^200000 calls	finished in 3.89 s	**PASS**

Q4. Exception-Safe CSV Transaction Splitter

CO-1, CO-4 | Bloom L5 Evaluate | File handling, CSV, exception handling, dictionaries, sorting

Problem

Read a transactions CSV, validate each row, write valid CREDIT and DEBIT rows to credit.csv and debit.csv, write rejected rows with a reason to error.csv, keep going after bad rows, and print each account's net change sorted by absolute value (descending).

Approach

- The file is **streamed** with csv.reader: rows are written out as they are read, and only a dictionary of per-account totals stays in memory. This is what keeps memory at O(a) even for 1,000,000 rows.
- `validate_row()` raises a custom `RowValidationError` whose message is the reject reason: wrong column count, missing id or account, unknown type, non-numeric, non-finite or non-positive amount, and timestamps that fail `datetime.strptime('%Y-%m-%dT%H:%M:%S')` (which also rejects impossible dates such as month 13).
- Each row is processed inside its own try/except, so one bad row is logged to error.csv (with its line number) and processing continues.
- Amounts use `decimal.Decimal`, so 0.1 + 0.2 is exactly 0.3 in the summary. File-level problems (missing file, empty file) are caught separately with a clear message.
- The summary is sorted by (-|net|, account id); the account id tie-break makes the output deterministic.

Complexity

Time O(r + a log a): one pass over r rows plus sorting a accounts (within the required O(r log a)). **Space** O(a).

Assumptions: the type is case-insensitive (credit is accepted and written as CREDIT); output files are written next to the input file unless a second argument names a folder; duplicate transaction ids are not checked, because that would need O(r) memory and break the O(a) space requirement.

How to run

```
python ENROLLMENT_1_Q4.py transactions.csv [output_folder]
```

Source code — ENROLLMENT_1_Q4.py

```python
"""
Q4. Exception-Safe CSV Transaction Splitter
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-4

Reads a transaction CSV (tid, acc, type, amount, time) row by row and
    * writes valid CREDIT rows  -> credit.csv
    * writes valid DEBIT rows   -> debit.csv
    * writes rejected rows      -> error.csv   (with line number and reason)
    * prints account-wise net change, sorted by |net change| descending
      (ties broken by account id so the output is deterministic).

Every row is validated inside try/except, so one bad row never stops the run.
Amounts use decimal.Decimal, so 0.1 + 0.2 is exactly 0.3 (no float drift).
The file is streamed: only the per-account totals are kept in memory.

Run   : python ENROLLMENT_1_Q4.py transactions.csv [output_folder]
        (output folder defaults to the folder of the input file)
Time  : O(r + a log a)     Space : O(a)   (r = rows, a = accounts)
"""

import csv
import os
import sys
from collections import defaultdict
from datetime import datetime
from decimal import Decimal, InvalidOperation

EXPECTED_COLUMNS = 5
VALID_TYPES = ("CREDIT", "DEBIT")
TIME_FORMAT = "%Y-%m-%dT%H:%M:%S"


class RowValidationError(Exception):
    """Raised when a single CSV row is invalid; message is the reject reason."""


def validate_row(row):
    """Return a cleaned (tid, acc, type, amount, time) tuple or raise."""
    if len(row) != EXPECTED_COLUMNS:
        raise RowValidationError(f"expected {EXPECTED_COLUMNS} columns, found {len(row)}")
    tid, acc, ttype, amount_text, time_text = (field.strip() for field in row)

    if not tid:
        raise RowValidationError("missing transaction_id")
    if not acc:
        raise RowValidationError("missing account_id")

    ttype = ttype.upper()
    if ttype not in VALID_TYPES:
        raise RowValidationError(f"invalid type '{ttype}'")

    try:
        amount = Decimal(amount_text)
    except InvalidOperation:
        raise RowValidationError(f"amount '{amount_text}' is not numeric") from None
    if not amount.is_finite():
        raise RowValidationError(f"amount '{amount_text}' is not a finite number")
    if amount <= 0:
        raise RowValidationError("amount must be greater than 0")

    try:
        datetime.strptime(time_text, TIME_FORMAT)
    except ValueError:
        raise RowValidationError(f"invalid timestamp '{time_text}'") from None

    return tid, acc, ttype, amount, time_text


def format_amount(value):
    """500 -> '500', 12.50 -> '12.5', -100 -> '-100'."""
    if value == value.to_integral_value():
```

```python
        return str(value.quantize(Decimal(1)))
    return format(value.normalize(), "f")


def split_transactions(input_path, out_dir):
    """Process the file. Returns (net_by_account dict, counts dict)."""
    net = defaultdict(Decimal)
    counts = {"CREDIT": 0, "DEBIT": 0, "ERROR": 0}

    with open(input_path, newline="", encoding="utf-8-sig") as src, \
         open(os.path.join(out_dir, "credit.csv"), "w", newline="", encoding="utf-8") as cf, \
         open(os.path.join(out_dir, "debit.csv"), "w", newline="", encoding="utf-8") as df, \
         open(os.path.join(out_dir, "error.csv"), "w", newline="", encoding="utf-8") as ef:

        reader = csv.reader(src)
        writers = {"CREDIT": csv.writer(cf), "DEBIT": csv.writer(df)}
        error_writer = csv.writer(ef)

        header = next(reader, None)
        if header is None:
            raise ValueError("input file is empty (no header row)")
        clean_header = [h.strip() for h in header]
        for w in writers.values():
            w.writerow(clean_header)
        error_writer.writerow(["line_no"] + clean_header + ["reason"])

        for line_no, row in enumerate(reader, start=2):
            if not any(field.strip() for field in row):
                continue                                    # skip blank lines
            try:
                tid, acc, ttype, amount, time_text = validate_row(row)
            except RowValidationError as err:
                counts["ERROR"] += 1
                error_writer.writerow([line_no] + row + [str(err)])
                continue
            writers[ttype].writerow([tid, acc, ttype, format_amount(amount), time_text])
            counts[ttype] += 1
            net[acc] += amount if ttype == "CREDIT" else -amount
    return net, counts


def main():
    if len(sys.argv) < 2:
        print("Usage: python ENROLLMENT_1_Q4.py <transactions.csv> [output_folder]")
        sys.exit(1)
    input_path = sys.argv[1]
    out_dir = sys.argv[2] if len(sys.argv) > 2 else (os.path.dirname(input_path) or ".")

    try:
        os.makedirs(out_dir, exist_ok=True)
        net, counts = split_transactions(input_path, out_dir)
    except FileNotFoundError:
        print(f"ERROR: file not found: {input_path}")
        sys.exit(1)
    except (OSError, ValueError, csv.Error) as err:
        print(f"ERROR: {err}")
        sys.exit(1)

    for acc, change in sorted(net.items(), key=lambda kv: (-abs(kv[1]), kv[0])):
        print(acc, format_amount(change))
    print(f"Files created: credit.csv ({counts['CREDIT']} rows), "
          f"debit.csv ({counts['DEBIT']} rows), error.csv ({counts['ERROR']} rows)")


if __name__ == "__main__":
    main()
```

Test cases — 6 of 6 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	transactions.csv: tid,acc,type,amount,time T1,A1,CREDIT,500,2026-01-01T10:00:00 T2,A1,DEBIT,abc,2026-01-01T10:05:00 T3,A2,DEBIT,100,2026-01-01T10:10:00	A1 500 A2 -100 Files created: credit.csv (1 rows), debit.csv (1 rows), error.csv (1 rows) --- error.csv --- line_no,tid,acc,type,amount,time,reason 3,T2,A1,DEBIT,abc,2026-01-01T10:05:00,amount 'abc' is not numeric	PASS
2	**Messy file: 7 different invalid rows, decimals, blank line** custom	transactions.csv: tid,acc,type,amount,time T1,A1,CREDIT,500,2026-01-01T10:00:00 T2,A1,DEBIT,abc,2026-01-01T10:05:00 T3,A2,DEBIT,100,2026-01-01T10:10:00 T4,A3,credit,0.1,2026-02-01T09:00:00 T5,A3,CREDIT,0.2,2026-02-01T09:01:00 T6,A2,DEBIT,-5,2026-02-01T09:02:00 T7,A4,REFUND,10,2026-02-01T09:03:00 T8,A4,CREDIT,10,2026-13-01T09:03:00 T9,A5,CREDIT,10 T10,,DEBIT,5,2026-02-01T09:04:00 T11,A1,DEBIT,200.50,2026-02-02T10:00:00 T12,A5,CREDIT,NaN,2026-02-02T10:00:00	A1 299.5 A2 -100 A3 0.3 Files created: credit.csv (3 rows), debit.csv (2 rows), error.csv (7 rows) --- error.csv --- line_no,tid,acc,type,amount,time,reason 3,T2,A1,DEBIT,abc,2026-01-01T10:05:00,amount 'abc' is not numeric 7,T6,A2,DEBIT,-5,2026-02-01T09:02:00,amount must be greater than 0 8,T7,A4,REFUND,10,2026-02-01T09:03:00,invalid type 'REFUND' 9,T8,A4,CREDIT,10,2026-13-01T09:03:00,invalid timestamp '2026-13-01T09:03:00' 10,T9,A5,CREDIT,10,"expected 5 columns, found 4" 12,T10,,DEBIT,5,2026-02-01T09:04:00,missing account_id 14,T12,A5,CREDIT,NaN,2026-02-02T10:00:00,amount 'NaN' is not a finite number	PASS
3	**Boundary: header only, no records** boundary	transactions.csv: tid,acc,type,amount,time	Files created: credit.csv (0 rows), debit.csv (0 rows), error.csv (0 rows)	PASS
4	**Invalid: completely empty file** invalid	transactions.csv:	ERROR: input file is empty (no header row)	PASS
5	**Invalid: file does not exist** invalid	/no/such/file.csv	ERROR: file not found: /no/such/file.csv	PASS
6	**Max size: 1,000,000 rows, 5000 accounts** performance	1,000,000 random rows, every 1000th invalid	finished in 10.52 s	PASS

Q5. Object-Oriented Bank Settlement System

CO-1, CO-2 | Bloom L6 Create | OOP, encapsulation, custom exceptions, rollback, dictionaries

Problem

Design Account, Transaction and Bank classes supporting deposit, withdraw and transfer, with transaction history, no overdraft, and batches that are rolled back completely if any operation inside them fails. Use custom exceptions and properties.

Approach

- **Encapsulation:** `Account.balance` is a read-only property; the balance can only change through `deposit()` and `withdraw()`, and `withdraw()` refuses to overdraw. `history` returns a copy (tuple) so callers cannot edit it. `Transaction` objects are immutable.
- **Custom exceptions** form a hierarchy under `BankError`: AccountNotFoundError, InvalidAmountError, InsufficientFundsError and BatchError, so the driver can catch them all in one place.
- **Rollback with an undo log:** while a batch is open, the first time an account is touched its (balance, history length) is saved in a dictionary. If an operation fails, the rest of the batch is skipped, and at BATCH_END every saved account is restored. Extra memory is O(batch size), not a copy of all accounts.
- Transfer checks the source balance before changing anything, so a failed transfer outside a batch never leaves money half-moved.
- Accounts live in a dictionary for O(1) lookup and are sorted once at the end for output.

Complexity

Time O(q + n log n): every operation is O(1) average (dictionary lookups, undo entries); the final sort is n log n. **Space** O(n + history + batch size).

Assumptions: a failed operation outside a batch is skipped and reported on stderr only; a batch still open at end of input counts as failed; a nested BATCH_BEGIN makes the current batch fail; account ids are printed in lexicographic order.

How to run

```
python ENROLLMENT_1_Q5.py < sample_input.txt
```

Source code — ENROLLMENT_1_Q5.py

```python
"""
Q5. Object-Oriented Bank Settlement System
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-2

Classes
    Account      - id, balance (read-only property), transaction history
    Transaction  - one immutable entry in an account's history
    Bank         - owns accounts (dict), executes operations, manages batches
    BankError and subclasses - custom exceptions for every failure type

Batch rollback (undo log)
    While a batch is open, before an account is changed for the first time
    in that batch, its balance and history length are saved in an undo log.
    If any operation in the batch fails, the remaining operations of that
    batch are skipped and every saved account is restored -> the whole batch
    has no effect. On BATCH_END with no failure, the undo log is discarded.
    Extra memory is O(batch_size).

Assumptions
    * A failed operation OUTSIDE a batch is atomic on its own: it is skipped
      and reported on stderr; it does not appear in the required output.
    * A batch that is still open at end of input is treated as failed.
    * Nested BATCH_BEGIN makes the current batch fail.
    * Account ids are printed in lexicographic order.

Run   : python ENROLLMENT_1_Q5.py < input.txt
Time  : O(q + n log n)     Space : O(n + history + batch_size)
"""

import sys

MAX_AMOUNT = 10 ** 9


# ------------------------- custom exceptions -------------------------
class BankError(Exception):
    """Base class for every bank-related error."""


class AccountNotFoundError(BankError):
    pass


class InvalidAmountError(BankError):
    pass


class InsufficientFundsError(BankError):
    pass


class BatchError(BankError):
    pass


# ------------------------------ models ------------------------------
class Transaction:
    """Immutable history entry."""

    __slots__ = ("_kind", "_amount", "_counterparty")

    def __init__(self, kind, amount, counterparty=None):
        self._kind = kind
        self._amount = amount
        self._counterparty = counterparty

    @property
    def kind(self):
        return self._kind

    @property
```

```python
    def amount(self):
        return self._amount

    def __repr__(self):
        extra = f" ({self._counterparty})" if self._counterparty else ""
        return f"{self._kind} {self._amount}{extra}"


class Account:
    """A bank account. Balance can only change through deposit/withdraw."""

    def __init__(self, account_id, opening_balance=0):
        if opening_balance < 0:
            raise InvalidAmountError(f"opening balance of {account_id} is negative")
        self._id = account_id
        self._balance = opening_balance
        self._history = []

    @property
    def account_id(self):
        return self._id

    @property
    def balance(self):              # read-only: no setter on purpose
        return self._balance

    @property
    def history(self):
        return tuple(self._history)  # copy, so callers cannot modify it

    def deposit(self, amount, note=None):
        self._balance += amount
        self._history.append(Transaction("DEPOSIT", amount, note))

    def withdraw(self, amount, note=None):
        if amount > self._balance:
            raise InsufficientFundsError(
                f"{self._id} has {self._balance}, cannot withdraw {amount}")
        self._balance -= amount
        self._history.append(Transaction("WITHDRAW", amount, note))

    # -- used only by Bank for rollback --
    def _snapshot(self):
        return self._balance, len(self._history)

    def _restore(self, snap):
        self._balance, hist_len = snap
        del self._history[hist_len:]


class Bank:
    def __init__(self):
        self._accounts = {}
        self._undo = None            # dict account_id -> snapshot, while a batch is open
        self._batch_no = 0
        self._batch_failed = False
        self.failed_batches = []

    # ---------- account management ----------
    def open_account(self, account_id, balance):
        if account_id in self._accounts:
            raise BankError(f"duplicate account {account_id}")
        self._accounts[account_id] = Account(account_id, balance)

    def _get(self, account_id):
        try:
            return self._accounts[account_id]
        except KeyError:
            raise AccountNotFoundError(f"account {account_id} does not exist") from None

    @staticmethod
    def _check_amount(amount):
```

```
145             if not 0 < amount <= MAX_AMOUNT:
146                 raise InvalidAmountError(f"amount {amount} must be in 1..{MAX_AMOUNT}")
147
148         def _remember(self, account):
149             """Save account state once per batch, before its first change."""
150             if self._undo is not None and account.account_id not in self._undo:
151                 self._undo[account.account_id] = account._snapshot()
152
153         # ---------- operations ----------
154         def deposit(self, acc_id, amount):
155             self._check_amount(amount)
156             acc = self._get(acc_id)
157             self._remember(acc)
158             acc.deposit(amount)
159
160         def withdraw(self, acc_id, amount):
161             self._check_amount(amount)
162             acc = self._get(acc_id)
163             self._remember(acc)
164             acc.withdraw(amount)
165
166         def transfer(self, src_id, dst_id, amount):
167             self._check_amount(amount)
168             src, dst = self._get(src_id), self._get(dst_id)
169             if src is dst:
170                 raise BankError("cannot transfer to the same account")
171             if amount > src.balance:          # check first, so nothing changes on failure
172                 raise InsufficientFundsError(
173                     f"{src_id} has {src.balance}, cannot transfer {amount}")
174             self._remember(src)
175             self._remember(dst)
176             src.withdraw(amount, note=f"to {dst_id}")
177             dst.deposit(amount, note=f"from {src_id}")
178
179         # ---------- batches ----------
180         @property
181         def in_batch(self):
182             return self._undo is not None
183
184         def begin_batch(self):
185             if self.in_batch:
186                 raise BatchError("nested BATCH_BEGIN is not allowed")
187             self._batch_no += 1
188             self._undo = {}
189             self._batch_failed = False
190
191         def end_batch(self):
192             if not self.in_batch:
193                 raise BatchError("BATCH_END without BATCH_BEGIN")
194             if self._batch_failed:
195                 self._rollback()
196             self._undo = None
197
198         def _rollback(self):
199             for acc_id, snap in self._undo.items():
200                 self._accounts[acc_id]._restore(snap)
201             self.failed_batches.append(self._batch_no)
202
203         def execute(self, op, args):
204             """Run one operation. Inside a failed batch, operations are skipped."""
205             if op == "BATCH_BEGIN":
206                 if self.in_batch:              # nested begin -> current batch fails
207                     self._batch_failed = True
208                     raise BatchError("nested BATCH_BEGIN is not allowed")
209                 return self.begin_batch()
210             if op == "BATCH_END":
211                 return self.end_batch()
212             if self.in_batch and self._batch_failed:
213                 return                         # batch already doomed: skip
214             try:
215                 handlers = {"DEPOSIT": (self.deposit, 2), "WITHDRAW": (self.withdraw, 2),
216                             "TRANSFER": (self.transfer, 3)}
```

```
            if op not in handlers:
                raise BankError(f"unknown operation '{op}'")
            func, argc = handlers[op]
            if len(args) != argc:
                raise BankError(f"{op} needs {argc} arguments")
            try:
                amount = int(args[-1])
            except ValueError:
                raise InvalidAmountError(f"amount '{args[-1]}' is not an integer") from None
            func(*args[:-1], amount)
        except BankError:
            if self.in_batch:
                self._batch_failed = True
            raise

    def finish(self):
        """Close a batch left open at end of input (treated as failed)."""
        if self.in_batch:
            self._batch_failed = True
            self.end_batch()

    def sorted_accounts(self):
        return [self._accounts[k] for k in sorted(self._accounts)]


# ------------------------------ driver ------------------------------
def solve(text):
    lines = [ln.split() for ln in text.splitlines() if ln.strip()]
    bank = Bank()
    n = int(lines[0][0])
    for parts in lines[1:n + 1]:
        if len(parts) != 2:
            raise BankError(f"bad account line: {' '.join(parts)}")
        bank.open_account(parts[0], int(parts[1]))
    q = int(lines[n + 1][0])
    for op_no, parts in enumerate(lines[n + 2:n + 2 + q], start=1):
        try:
            bank.execute(parts[0].upper(), parts[1:])
        except BankError as err:
            print(f"op {op_no}: {type(err).__name__}: {err}", file=sys.stderr)
    bank.finish()

    out = [f"FAILED {b}" for b in bank.failed_batches]
    out += [f"{acc.account_id} {acc.balance}" for acc in bank.sorted_accounts()]
    return "\n".join(out)


def main():
    try:
        print(solve(sys.stdin.read()))
    except (BankError, ValueError, IndexError) as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 7 of 7 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	3 A 1000 B 200 C 0 5 TRANSFER A B 300 BATCH_BEGIN WITHDRAW B 1000 DEPOSIT C 50 BATCH_END	FAILED 1 A 700 B 500 C 0	PASS

#	Test case	Input	Output (expected = actual)	Result
2	**Successful batch is committed** custom	2 A 100 B 0 4 BATCH_BEGIN TRANSFER A B 60 DEPOSIT A 10 BATCH_END	A 50 B 60	**PASS**
3	**Overdraft outside batch is rejected; withdraw to exactly 0 allowed** boundary	1 X 50 3 WITHDRAW X 60 WITHDRAW X 50 DEPOSIT X 5	X 5	**PASS**
4	**Second batch fails mid-way; partial transfer rolled back** custom	3 A 100 B 100 C 100 10 BATCH_BEGIN DEPOSIT A 1 BATCH_END BATCH_BEGIN TRANSFER A B 50 TRANSFER B C 500 DEPOSIT C 1000 BATCH_END DEPOSIT Z 5 WITHDRAW C 100	FAILED 2 A 101 B 100 C 0	**PASS**
5	**Batch never closed -> treated as failed** boundary	1 A 10 2 BATCH_BEGIN DEPOSIT A 5	FAILED 1 A 10	**PASS**
6	**Negative amount inside batch** invalid	2 A 10 B 0 4 BATCH_BEGIN DEPOSIT B 7 DEPOSIT A -5 BATCH_END	FAILED 1 A 10 B 0	**PASS**
7	**Max size: 100000 accounts, ~300000 operations** performance	random deposits + small transfer batches	finished in 2.09 s	**PASS**

Q6. Python Module Dependency Resolver

CO-1, CO-2 | Bloom L4 Analyze | Graphs, topological sort, heaps, sets, cycle detection

Problem

Given modules and 'a imports b' relations, print a loading order (lexicographically smallest when there is a choice), ignoring duplicate edges; if there is a circular dependency, print CYCLE and one cycle.

Approach

- 'a imports b' means b must load first, so the edge is b → a. Duplicate import lines are removed by storing edges in a **set**.
- **Kahn's algorithm**: indegree[x] counts the modules x imports. All modules with indegree 0 are kept in a **min-heap** (heapq); repeatedly popping the smallest gives the lexicographically smallest valid order.
- If fewer than n modules are output, a cycle exists. Every remaining module still has a remaining dependency, so walking from one remaining module to one of its remaining dependencies must eventually revisit a module; the walk from that point on is a cycle. This takes O(n + e) with no recursion.
- The cycle is printed as `a -> b -> c -> a`, read as 'a imports b, b imports c, c imports a'.
- Validation: counts within limits, duplicate module names, name length, and edges that mention unknown modules.

Complexity

Time O((n + e) log n): each module is pushed and popped from the heap once and each edge is relaxed once. **Space** O(n + e) for the adjacency lists.

How to run

```
python ENROLLMENT_1_Q6.py < sample_input.txt
```

Source code — ENROLLMENT_1_Q6.py

```python
"""
Q6. Python Module Dependency Resolver
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-2

"a imports b" means b must be loaded before a  ->  directed edge b -> a.

Algorithm: Kahn's topological sort with a min-heap
    * indegree[x] = number of distinct modules x imports
    * all modules with indegree 0 go into a heap (heapq)
    * repeatedly pop the lexicographically smallest ready module, output it,
      and decrease the indegree of the modules that import it
    * duplicate edges are removed with a set, so they are counted once

Cycle reporting
    If fewer than n modules were output, the remaining ones all still have
    an unloaded dependency. Starting from any remaining module and always
    stepping to one of its remaining dependencies, we must eventually revisit
    a module; the part of the walk from that module onward is a cycle.
    The cycle is printed as  a -> b -> c -> a  meaning "a imports b, ...".

Run   : python ENROLLMENT_1_Q6.py < input.txt
Time  : O((n + e) log n)     Space : O(n + e)
"""

import heapq
import sys


class InputError(Exception):
    pass


def read_graph(text):
    lines = [ln.split() for ln in text.splitlines() if ln.strip()]
    if not lines or len(lines[0]) != 2:
        raise InputError("first line must be: n e")
    try:
        n, e = int(lines[0][0]), int(lines[0][1])
    except ValueError:
        raise InputError("n and e must be integers") from None
    if not (1 <= n <= 200000 and 0 <= e <= 500000):
        raise InputError("n or e outside constraints")
    if len(lines) < 1 + n + e:
        raise InputError("not enough lines for modules and edges")

    modules = []
    seen = set()
    for parts in lines[1:1 + n]:
        name = parts[0]
        if len(parts) != 1 or len(name) > 50:
            raise InputError(f"bad module name line: {' '.join(parts)}")
        if name in seen:
            raise InputError(f"duplicate module '{name}'")
        seen.add(name)
        modules.append(name)

    edges = set()                       # (importer, dependency); set removes duplicates
    for parts in lines[1 + n:1 + n + e]:
        if len(parts) != 2:
            raise InputError(f"bad edge line: {' '.join(parts)}")
        a, b = parts
        if a not in seen or b not in seen:
            raise InputError(f"edge uses unknown module: {a} {b}")
        edges.add((a, b))
    return modules, edges


def resolve(modules, edges):
    """Return ('ORDER', list) or ('CYCLE', list_of_modules_in_cycle)."""
    dependents = {m: [] for m in modules}     # b -> modules that import b
    depends_on = {m: [] for m in modules}     # a -> modules that a imports
```

```python
    indegree = dict.fromkeys(modules, 0)
    for a, b in edges:
        dependents[b].append(a)
        depends_on[a].append(b)
        indegree[a] += 1

    heap = [m for m in modules if indegree[m] == 0]
    heapq.heapify(heap)
    order = []
    while heap:
        mod = heapq.heappop(heap)
        order.append(mod)
        for nxt in dependents[mod]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                heapq.heappush(heap, nxt)

    if len(order) == len(modules):
        return "ORDER", order
    return "CYCLE", find_cycle(indegree, depends_on)


def find_cycle(indegree, depends_on):
    """Walk dependency edges among unloaded modules until one repeats."""
    loaded = {m for m, d in indegree.items() if d == 0}
    start = min(m for m in indegree if m not in loaded)   # deterministic start
    position = {}                      # module -> index in walk
    walk = []
    cur = start
    while cur not in position:
        position[cur] = len(walk)
        walk.append(cur)
        # pick the smallest still-unloaded dependency (always exists)
        cur = min(d for d in depends_on[cur] if d not in loaded)
    cycle = walk[position[cur]:]
    return cycle + [cycle[0]]


def solve(text):
    modules, edges = read_graph(text)
    kind, result = resolve(modules, edges)
    if kind == "ORDER":
        return " ".join(result)
    return "CYCLE\n" + " -> ".join(result)


def main():
    try:
        print(solve(sys.stdin.read()))
    except InputError as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 7 of 7 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	4 3 core db ui auth ui auth auth db db core	core db auth ui	PASS

#	Test case	Input	Output (expected = actual)	Result
2	**Lexicographic choice + duplicate edge ignored** custom	4 3 zeta alpha mid beta mid alpha mid alpha beta zeta	alpha mid zeta beta	**PASS**
3	**Cycle a -> b -> c -> a** custom	4 4 a b c d a b b c c a d a	CYCLE a -> b -> c -> a	**PASS**
4	**Self import** boundary	2 1 x y y y	CYCLE y -> y	**PASS**
5	**Boundary: one module, no edges** boundary	1 0 solo	solo	**PASS**
6	**Invalid: edge with unknown module** invalid	1 1 a a ghost	INPUT ERROR: edge uses unknown module: a ghost	**PASS**
7	**Max size: n=200000, e=500000 (random DAG)** performance	random acyclic import graph	finished in 3.48 s	**PASS**

Q7. Interactive Formula Validator with Custom Exceptions

CO-1, CO-2 | Bloom L5 Evaluate | Exception handling, assertions, dictionaries, parsing

Problem

An interactive calculator for 'operand operator operand' formulas and 'name = value' assignments. Operands are numbers or stored variables; operators are + - * / %. Bad format, unknown variables, division by zero and unsupported operators each raise their own custom exception, and the program continues until quit.

Approach

- One regular expression splits a formula into operand, operator and operand; the operator group accepts any run of symbols, so `**` and `//` are recognised and reported as unsupported rather than as bad format.
- Custom exceptions share a base class `CalculatorError`: InvalidFormatError, UnknownVariableError, DivisionByZeroError, UnsupportedOperatorError and NumericOverflowError. The main loop catches the base class, prints `ExceptionName: message` and continues.
- Variables live in a dictionary (O(1) average lookup). Names follow Python identifier rules and keywords such as `if` are rejected. Assignments may take a formula (`y = x * 2`).
- Operators map to functions from the `operator` module through a dictionary instead of an if-chain. An `assert` documents the invariant that only validated operators are applied.
- Integers stay integers (15, not 15.0); a prompt is shown only when input comes from a keyboard, so redirected test files produce clean output.

Complexity

Time O(1) average per line (one regex match plus dictionary lookups). **Space** O(v) for v stored variables.

> The sample output in the paper shows only the exception name. As the problem statement also asks for 'a meaningful message', each error line is printed as `DivisionByZeroError: cannot compute x / 0: divisor is zero`.

> A negative operand needs a space after the operator: `5 - -3` works, while `5--3` is read as the unsupported operator `--`.

How to run

```
python ENROLLMENT_1_Q7.py (type formulas, end with quit)
```

Source code — ENROLLMENT_1_Q7.py

```python
"""
Q7. Interactive Formula Validator with Custom Exceptions
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-2

Accepted lines
    <operand> <operator> <operand>          e.g.  x + 5,  3.5*2,  y % 4
    <name> = <operand>                      e.g.  x = 10
    <name> = <operand> <operator> <operand> e.g.  y = x * 2   (stores result)
    quit                                    ends the program (EOF also ends it)

Operands  : integers, decimals (optionally signed) or stored variables.
Operators : + - * / %
Errors    : each problem raises its own custom exception, is caught in the
            main loop and printed as  ExceptionName: message , then the
            calculator continues with the next line.

    InvalidFormatError        line is not "operand operator operand"
    UnknownVariableError      operand is a name that was never assigned
    DivisionByZeroError       x / 0 or x % 0
    UnsupportedOperatorError  e.g. **, //, ^
    NumericOverflowError      result is not a finite float

Negative operands must be separated by a space from the operator:
"5 - -3" is valid, while "5--3" is read as operator "--" (unsupported).

Run   : python ENROLLMENT_1_Q7.py           (type lines, finish with quit)
        python ENROLLMENT_1_Q7.py < input.txt
Time  : O(1) average per line (one regex match + dict lookups)
Space : O(v) for v stored variables
"""

import keyword
import math
import operator
import re
import sys


# --------------------------- custom exceptions ---------------------------
class CalculatorError(Exception):
    """Base class for every calculator error."""


class InvalidFormatError(CalculatorError):
    pass


class UnknownVariableError(CalculatorError):
    pass


class DivisionByZeroError(CalculatorError):
    pass


class UnsupportedOperatorError(CalculatorError):
    pass


class NumericOverflowError(CalculatorError):
    pass


# ------------------------------ parsing ------------------------------
OPERAND = r"[+-]?(?:\d+\.?\d*|\.\d+)(?:[eE][+-]?\d+)?|[A-Za-z_]\w*"
FORMULA_RE = re.compile(rf"^\s*({OPERAND})\s*([^\w\s.]+)\s*({OPERAND})\s*$")
ASSIGN_RE = re.compile(r"^\s*(\S+?)\s*=(?!=)\s*(.+?)\s*$")   # '==' is not assignment
NUMBER_RE = re.compile(r"^[+-]?(?:\d+\.?\d*|\.\d+)(?:[eE][+-]?\d+)?$")

OPERATIONS = {
    "+": operator.add,
```

```
    "-": operator.sub,
    "*": operator.mul,
    "/": operator.truediv,
    "%": operator.mod,
}


class Calculator:
    def __init__(self):
        self.variables = {}                         # name -> int/float

    @staticmethod
    def is_valid_name(name):
        """Python identifier rules, and not a keyword such as 'if'."""
        return name.isidentifier() and not keyword.iskeyword(name) and name != "quit"

    def operand_value(self, token):
        if NUMBER_RE.match(token):
            is_int = not any(c in token for c in ".eE")
            return int(token) if is_int else float(token)
        if token not in self.variables:
            raise UnknownVariableError(f"variable '{token}' is not defined")
        return self.variables[token]

    def evaluate(self, text):
        """Evaluate a formula or a single operand."""
        text = text.strip()
        if re.fullmatch(OPERAND, text):             # plain value, used in x = 10
            return self.operand_value(text)
        m = FORMULA_RE.match(text)
        if not m:
            raise InvalidFormatError(f"'{text}' is not of the form: operand operator operand")
        left, op, right = m.groups()
        if op not in OPERATIONS:
            raise UnsupportedOperatorError(f"operator '{op}' is not supported (use + - * / %)")
        a, b = self.operand_value(left), self.operand_value(right)
        if op in "/%" and b == 0:
            raise DivisionByZeroError(f"cannot compute {left} {op} {right}: divisor is zero")
        assert op in OPERATIONS, "operator must be validated before use"
        try:
            result = OPERATIONS[op](a, b)
        except OverflowError:
            raise NumericOverflowError("result is too large") from None
        if isinstance(result, float) and not math.isfinite(result):
            raise NumericOverflowError("result is outside the float range")
        return result

    def process(self, line):
        """Handle one input line. Returns text to print, or None."""
        m = ASSIGN_RE.match(line)
        if m:
            name, expr = m.groups()
            if not self.is_valid_name(name):
                raise InvalidFormatError(f"'{name}' is not a valid variable name")
            self.variables[name] = self.evaluate(expr)
            return None                             # assignments print nothing
        return format_number(self.evaluate(line))


def format_number(value):
    """15 -> '15', 2.5 -> '2.5', 1/3 -> '0.3333333333'."""
    if isinstance(value, float):
        if value.is_integer() and abs(value) < 1e15:
            return str(int(value))
        return f"{value:.10g}"
    return str(value)


def run(lines, out=print):
    calc = Calculator()
    for raw in lines:
        line = raw.strip()
```

```
        if not line:
            continue
        if line.lower() == "quit":
            break
        try:
            result = calc.process(line)
            if result is not None:
                out(result)
        except CalculatorError as err:
            out(f"{type(err).__name__}: {err}")


def interactive_lines():
    """Yield lines from the keyboard (with a prompt) or from a redirected file."""
    prompt = "calc> " if sys.stdin.isatty() else ""
    while True:
        try:
            yield input(prompt)
        except EOFError:
            return


if __name__ == "__main__":
    run(interactive_lines())
```

Test cases — 5 of 5 passed

#	Test case	Input	Output (expected = actual)	Result

1	**Sample test**
	sample
		`x = 10`
		`x + 5`
		`x / 0`
		`3 ** 2`
		`quit`
			`15`
			`DivisionByZeroError: cannot compute x / 0: divisor is zero`
			`UnsupportedOperatorError: operator '** ' is not supported (use + - * / %)`
				PASS

2	**Decimals, %, unknown variable, bad format, assignment from formula, quit**
	custom
		`a = 7`
		`b = 2.5`
		`a % 3`
		`a / 2`
		`b * a`
		`y + 1`
		`3 +`
		`c = a * 2`
		`c - -4`
		`quit`
		`1 + 1`
			`1`
			`3.5`
			`17.5`
			`UnknownVariableError: variable 'y' is not defined`
			`InvalidFormatError: '3 +' is not of the form: operand operator operand`
			`18`
				PASS

3	**Invalid names, //, modulo by zero, float overflow**
	invalid
		`2x = 5`
		`if = 3`
		`10 // 3`
		`5 % 0`
		`1e308 * 10`
		`quit`
			`InvalidFormatError: '2x' is not a valid variable name`
			`InvalidFormatError: 'if' is not a valid variable name`
			`UnsupportedOperatorError: operator '// ' is not supported (use + - * / %)`
			`DivisionByZeroError: cannot compute 5 % 0: divisor is zero`
			`NumericOverflowError: result is outside the float range`
				PASS

4	**Boundary: no spaces, leading-dot decimal, EOF without quit**
	boundary
		`v=.5`
		`v*4`
		`0-0`
			`2`
			`0`
				PASS

5	**Max size: 100000 lines** performance	`50000 assignments + 50000 formulas`	`finished in 1.70 s`	**PASS**

Q8. Compressed Log Index using Pickle and Zip

CO-1, CO-4 | Bloom L6 Create | File handling, pickling, zip, dictionaries, indexing

Problem

BUILD: scan every log file in a folder, build an inverted index token → (file, line), store it with pickle and zip it together with the logs. SEARCH: load the pickled index and print every file:line match for each query token.

Approach

- Files are found with `os.walk` (sub-folders included) and sorted, so file ids are stable. Lines are read one at a time with `errors='replace'`, so a stray binary byte does not stop the build.
- Tokens are normalised to lower case runs of letters, digits and underscore (ERROR, `error` and `Error` are one token); tokens longer than 50 characters are skipped. A token is stored once per line even if it repeats on that line.
- Postings are packed into `array('Q')` as `file_id * 2**32 + line`: 8 bytes per posting instead of a Python tuple of about 64 bytes. This is what makes 5,000,000 lines practical in memory.
- The index is pickled to `<name>_index.pkl` and written into a ZIP_DEFLATED archive with all logs under `logs/`. SEARCH accepts either the .pkl or the .zip.
- **Security:** unpickling can run arbitrary code, so SEARCH uses a restricted Unpickler that only allows the array type the index contains. A test pickles an `os.system` call and confirms it is blocked.

Complexity

Time O(total characters in all files) for BUILD; SEARCH is O(matches) per token after loading. **Space** O(unique tokens + postings).

BUILD can be run with arguments (`BUILD logs archive.zip`) or the same words on standard input. For SEARCH via standard input: line 1 `SEARCH index.pkl`, line 2 q, then q tokens.

How to run

```
python ENROLLMENT_1_Q8.py BUILD logs archive.zip
python ENROLLMENT_1_Q8.py SEARCH archive_index.pkl error timeout
```

Source code — ENROLLMENT_1_Q8.py

```python
"""
Q8. Compressed Log Index using Pickle and Zip
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-4

BUILD  folder archive.zip
    * scans every regular file in the folder (sub-folders included)
    * normalises tokens: lower-case runs of letters/digits/underscore,
      length 1..50 (longer tokens are ignored)
    * inverted index: token -> postings (file, line). A token appearing
      several times on one line is stored once for that line.
    * postings are packed into array('Q') as  file_id * 2**32 + line_no
      (8 bytes per posting instead of a ~64 byte tuple) -> fits 5M lines
    * saves  { "files": [...], "index": {...} }  with pickle to
      <archive>_index.pkl, then writes a ZIP_DEFLATED archive that contains
      all log files (under logs/) and index.pkl
    * prints FILES, LINES, TOKENS

SEARCH  index.pkl|archive.zip  token1 token2 ...
    * loads the pickle (directly, or index.pkl from inside the zip)
    * prints every file:line match per token

Security: pickle can run code while loading, so a restricted Unpickler is
used that only allows the array type our own index contains.

Run   : python ENROLLMENT_1_Q8.py BUILD logs archive.zip
        python ENROLLMENT_1_Q8.py SEARCH archive_index.pkl error timeout
        (with no arguments the same words are read from standard input:
         line 1 = mode line, for SEARCH line 2 = q, next q lines = tokens)
Time  : O(total characters)     Space : O(unique tokens + postings)
"""

import array
import io
import os
import pickle
import re
import sys
import zipfile

TOKEN_RE = re.compile(r"[a-z0-9_]+")
MAX_TOKEN_LEN = 50
LINE_BITS = 32


class LogIndexError(Exception):
    pass


# ------------------------------ BUILD ---------------------------------
def list_log_files(folder, skip_paths):
    """All regular files below folder, as sorted relative paths."""
    files = []
    for root, _dirs, names in os.walk(folder):
        for name in names:
            full = os.path.join(root, name)
            if os.path.abspath(full) in skip_paths or not os.path.isfile(full):
                continue
            files.append(os.path.relpath(full, folder).replace(os.sep, "/"))
    return sorted(files)


def build_index(folder, files):
    """Return (index dict, total_lines)."""
    index = {}
    total_lines = 0
    for file_id, rel in enumerate(files):
        path = os.path.join(folder, rel)
        try:
            with open(path, encoding="utf-8", errors="replace") as fh:
                for line_no, line in enumerate(fh, start=1):
                    total_lines += 1
```

```
                    key = (file_id << LINE_BITS) | line_no
                    for token in TOKEN_RE.findall(line.lower()):
                        if len(token) > MAX_TOKEN_LEN:
                            continue
                        postings = index.get(token)
                        if postings is None:
                            index[token] = array.array("Q", [key])
                        elif postings[-1] != key:     # once per line
                            postings.append(key)
        except OSError as err:
            print(f"WARNING: skipped {rel}: {err}", file=sys.stderr)
    return index, total_lines


def build(folder, zip_name):
    if not os.path.isdir(folder):
        raise LogIndexError(f"folder not found: {folder}")
    if not zip_name.lower().endswith(".zip"):
        zip_name += ".zip"
    pkl_name = zip_name[:-4] + "_index.pkl"
    skip = {os.path.abspath(zip_name), os.path.abspath(pkl_name)}

    files = list_log_files(folder, skip)
    if not files:
        raise LogIndexError("no files found in folder")
    index, total_lines = build_index(folder, files)

    payload = {"files": files, "index": index}
    with open(pkl_name, "wb") as fh:
        pickle.dump(payload, fh, protocol=pickle.HIGHEST_PROTOCOL)

    with zipfile.ZipFile(zip_name, "w", compression=zipfile.ZIP_DEFLATED) as zf:
        for rel in files:
            zf.write(os.path.join(folder, rel), arcname=f"logs/{rel}")
        zf.write(pkl_name, arcname="index.pkl")

    print(f"FILES {len(files)}")
    print(f"LINES {total_lines}")
    print(f"TOKENS {len(index)}")
    return pkl_name


# ------------------------------ SEARCH --------------------------------
class SafeUnpickler(pickle.Unpickler):
    """Only allow what our index actually contains."""

    ALLOWED = {("array", "array"), ("array", "_array_reconstructor")}

    def find_class(self, module, name):
        if (module, name) in self.ALLOWED:
            return super().find_class(module, name)
        raise pickle.UnpicklingError(f"blocked unsafe object {module}.{name}")


def load_index(path):
    if not os.path.isfile(path):
        raise LogIndexError(f"index file not found: {path}")
    try:
        if zipfile.is_zipfile(path):
            with zipfile.ZipFile(path) as zf:
                data = zf.read("index.pkl")
        else:
            with open(path, "rb") as fh:
                data = fh.read()
        payload = SafeUnpickler(io.BytesIO(data)).load()
    except (KeyError, pickle.UnpicklingError, EOFError) as err:
        raise LogIndexError(f"not a valid index: {err}") from None
    if not isinstance(payload, dict) or {"files", "index"} - payload.keys():
        raise LogIndexError("not a valid index file")
    return payload


```

```python
def search(index_path, tokens):
    payload = load_index(index_path)
    files, index = payload["files"], payload["index"]
    mask = (1 << LINE_BITS) - 1
    for raw in tokens:
        token = raw.lower()
        postings = index.get(token, ())
        print(f"[{token}] {len(postings)} match(es)")
        for key in postings:
            print(f"  {files[key >> LINE_BITS]}:{key & mask}")


# ------------------------------- main ---------------------------------
def read_args_from_stdin():
    lines = [ln.strip() for ln in sys.stdin.read().splitlines() if ln.strip()]
    if not lines:
        return []
    args = lines[0].split()
    if args and args[0].upper() == "SEARCH" and len(lines) > 1:
        q = int(lines[1])
        args += lines[2:2 + q]
    return args


def main():
    args = sys.argv[1:] or read_args_from_stdin()
    try:
        if len(args) == 3 and args[0].upper() == "BUILD":
            build(args[1], args[2])
        elif len(args) >= 3 and args[0].upper() == "SEARCH":
            search(args[1], args[2:])
        else:
            print("Usage: BUILD <folder> <archive.zip>  |  SEARCH <index.pkl|archive.zip> <tokens...>")
            sys.exit(1)
    except (LogIndexError, ValueError) as err:
        print(f"ERROR: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 7 of 7 passed

#	Test case	Input	Output (expected = actual)	Result

1	**BUILD on 3 files (one in a sub-folder)**
	sample
		`BUILD logs archive.zip`
		`app.log:`
		`  INFO server started on port 8080`
		`  ERROR Database timeout`
		`  WARN retry db_conn`
		`  ERROR error ERROR repeated on one li`
		`  ne`
		`auth.log:`
		`  INFO user login ok`
		`  ERROR invalid password for user admi`
		`  n`
		`old/2025.log:`
		`  Timeout while connecting`
			`FILES 3`
			`LINES 7`
			`TOKENS 24`
				PASS

2	**Zip contains all logs + index.pkl**
	custom
		`list archive.zip`	`index.pkl`
			`logs/app.log`
			`logs/auth.log`
			`logs/old/2025.log`
				PASS

3	**SEARCH: case-insensitive, once per line, missing token**
	custom
		`SEARCH archive_index.pkl ERROR timeout`
		`  nothing`
			`[error] 3 match(es)`
			`  app.log:2`
			`  app.log:4`
			`  auth.log:2`
			`[timeout] 2 match(es)`
			`  app.log:2`
			`  old/2025.log:1`
			`[nothing] 0 match(es)`
				PASS

4	**SEARCH via stdin, reading index from inside the zip**
	boundary
		`SEARCH archive.zip`
		`1`
		`user`
			`[user] 2 match(es)`
			`  auth.log:1`
			`  auth.log:2`
				PASS

#	Test case	Input	Output (expected = actual)	Result
5	**Malicious pickle is blocked** invalid	`SEARCH evil.pkl x  (pickle runs os.system)`	`ERROR: not a valid index: blocked unsafe object posix.system`	**PASS**
6	**Invalid: folder does not exist** invalid	`BUILD missing_dir a.zip`	`ERROR: folder not found: missing_dir`	**PASS**
7	**Large: 100 files, 2,000,000 lines, 16M tokens** performance	`BUILD big big.zip`	`finished in 37.74 s`	**PASS**

Q9. Threaded Job Scheduler Simulation

CO-1, CO-2 | Bloom L5 Evaluate | Multithreading, priority queues, simulation, synchronisation

Problem

Simulate w worker threads executing jobs (arrival, id, priority, duration, resources). Higher priority runs first, equal priority in arrival order, assignment must be thread-safe; print each job's worker, start and finish time and the average waiting time.

Approach

- **Dispatcher (discrete-event simulation):** three heaps hold the ready jobs ordered by (-priority, arrival, input order), the running jobs ordered by finish time, and the idle workers ordered by number. At each event time, finished jobs release their worker and resources, newly arrived jobs join the ready heap, then jobs are started while a worker is idle and the best job's resources fit.
- **Strict priority:** if the best job does not fit in the free resources, lower-priority jobs do not jump ahead, so a large job is never starved (test 'Resource pool R=5').
- **Worker threads:** one `threading.Thread` per worker receives its assignments through its own `queue.Queue` (a thread-safe FIFO). Workers write to the shared report inside a `threading.Lock`, and a `None` sentinel stops each thread.
- Times come from the deterministic simulation, so the report is identical on every run even though the threads run concurrently; real sleeping threads would make the output depend on timing.

Complexity

Time O(n log n + n log w): every job enters and leaves the ready and running heaps once. **Space** O(n + w).

Assumptions: the paper does not give the size of the resource pool, so the first line may be `w n` or `w n R`, with R defaulting to 100 (the largest amount one job may request). The lowest-numbered free worker is chosen, which reproduces the sample. Output lines follow input order.

How to run

```
python ENROLLMENT_1_Q9.py < sample_input.txt
```

Source code — ENROLLMENT_1_Q9.py

```python
"""
Q9. Threaded Job Scheduler Simulation
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-2

Design (two layers)
1. Dispatcher = discrete-event simulation in simulated integer time.
   Heaps (heapq) used:
       ready    : (-priority, arrival_time, input_index, job)  -> best job on top
       running  : (finish_time, worker_no, resources)          -> next completion
       idle     : worker numbers                               -> lowest free worker
   At every event time t:
       a) finish all jobs with finish_time == t (free worker + resources)
       b) add all jobs with arrival_time == t to the ready heap
       c) while a worker is idle AND the best ready job's resources fit in
          the free resource pool -> start it on the lowest-numbered worker
   Strict priority: if the best job does not fit, lower-priority jobs do
   NOT jump ahead (prevents starvation of large jobs).

2. Execution = real worker threads (threading.Thread, one per worker).
   The dispatcher hands each assignment to that worker's queue.Queue
   (thread-safe FIFO). Each worker "executes" its jobs and records the
   result in a shared report protected by a threading.Lock. A sentinel
   (None) tells a worker to stop. Because the start/finish times come from
   the deterministic simulation, the report is the same on every run even
   though the threads run concurrently.

Assumptions
   * First line may be "w n" or "w n R"; R = total resource units shared by
     all workers (default 100, the largest any single job may request).
   * Same-priority jobs run in arrival order; equal arrival -> input order.
   * Report lines are printed in input order. Waiting = start - arrival.

Run   : python ENROLLMENT_1_Q9.py < input.txt
Time  : O(n log n + n log w)     Space : O(n + w)
"""

import heapq
import queue
import sys
import threading

DEFAULT_RESOURCES = 100


class InputError(Exception):
    pass


class Job:
    __slots__ = ("index", "arrival", "job_id", "priority", "duration", "resources",
                 "worker", "start", "finish")

    def __init__(self, index, arrival, job_id, priority, duration, resources):
        self.index = index
        self.arrival = arrival
        self.job_id = job_id
        self.priority = priority
        self.duration = duration
        self.resources = resources
        self.worker = self.start = self.finish = None


def parse(text):
    lines = [ln.split() for ln in text.splitlines() if ln.strip()]
    if not lines or len(lines[0]) not in (2, 3):
        raise InputError("first line must be: w n [R]")
    try:
        head = [int(x) for x in lines[0]]
    except ValueError:
        raise InputError("w, n, R must be integers") from None
    w, n = head[0], head[1]
```

```
    pool = head[2] if len(head) == 3 else DEFAULT_RESOURCES
    if not (1 <= w <= 64 and 1 <= n <= 200000 and pool >= 1):
        raise InputError("w or n outside constraints")
    if len(lines) - 1 < n:
        raise InputError(f"expected {n} job lines")

    jobs, ids = [], set()
    for i, parts in enumerate(lines[1:n + 1]):
        if len(parts) != 5:
            raise InputError(f"job line {i + 1}: need 5 fields")
        try:
            arrival, priority, duration, res = (int(parts[0]), int(parts[2]),
                                                int(parts[3]), int(parts[4]))
        except ValueError:
            raise InputError(f"job line {i + 1}: numeric field expected") from None
        job_id = parts[1]
        if job_id in ids:
            raise InputError(f"duplicate job id {job_id}")
        if arrival < 0 or not 1 <= duration <= 10 ** 6 or not 1 <= res <= 100:
            raise InputError(f"job {job_id}: value outside constraints")
        if res > pool:
            raise InputError(f"job {job_id} needs {res} resources but pool has {pool}")
        ids.add(job_id)
        jobs.append(Job(i, arrival, job_id, priority, duration, res))
    return w, pool, jobs


def simulate(w, pool, jobs, dispatch):
    """Event loop. dispatch(worker_no, job) is called for every start."""
    arrivals = sorted(jobs, key=lambda j: (j.arrival, j.index))
    ready, running = [], []
    idle = list(range(1, w + 1))
    free_res = pool
    a = 0                                   # next arrival pointer
    while a < len(arrivals) or ready or running:
        # next event time
        candidates = []
        if a < len(arrivals):
            candidates.append(arrivals[a].arrival)
        if running:
            candidates.append(running[0][0])
        t = min(candidates)

        while running and running[0][0] == t:          # a) completions
            _, worker_no, res = heapq.heappop(running)
            heapq.heappush(idle, worker_no)
            free_res += res
        while a < len(arrivals) and arrivals[a].arrival == t:   # b) arrivals
            job = arrivals[a]
            heapq.heappush(ready, (-job.priority, job.arrival, job.index, job))
            a += 1
        while ready and idle and ready[0][3].resources <= free_res:  # c) dispatch
            job = heapq.heappop(ready)[3]
            worker_no = heapq.heappop(idle)
            job.worker, job.start, job.finish = worker_no, t, t + job.duration
            free_res -= job.resources
            heapq.heappush(running, (job.finish, worker_no, job.resources))
            dispatch(worker_no, job)


class WorkerPool:
    """Real threads that receive assignments through thread-safe queues."""

    def __init__(self, w):
        self.queues = {k: queue.Queue() for k in range(1, w + 1)}
        self.report = []
        self.lock = threading.Lock()
        self.threads = [threading.Thread(target=self._work, args=(k,), daemon=True)
                        for k in self.queues]
        for th in self.threads:
            th.start()

```

```python
    def _work(self, worker_no):
        q = self.queues[worker_no]
        while True:
            job = q.get()
            if job is None:                        # sentinel: no more work
                break
            line = (job.index, f"{job.job_id} W{worker_no} {job.start} {job.finish}")
            with self.lock:                        # shared report -> critical section
                self.report.append(line)

    def dispatch(self, worker_no, job):
        self.queues[worker_no].put(job)

    def shutdown(self):
        for q in self.queues.values():
            q.put(None)
        for th in self.threads:
            th.join()


def solve(text):
    w, pool, jobs = parse(text)
    workers = WorkerPool(w)
    try:
        simulate(w, pool, jobs, workers.dispatch)
    finally:
        workers.shutdown()
    lines = [line for _, line in sorted(workers.report)]
    avg_wait = sum(j.start - j.arrival for j in jobs) / len(jobs)
    lines.append(f"AVG_WAIT {avg_wait:.2f}")
    return "\n".join(lines)


def main():
    try:
        print(solve(sys.stdin.read()))
    except InputError as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 6 of 6 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	2 3 0 J1 2 5 1 1 J2 5 3 1 2 J3 2 2 1	J1 W1 0 5 J2 W2 1 4 J3 W2 4 6 AVG_WAIT 0.67	PASS
2	**One worker, priority order with later arrivals** custom	1 4 0 A 1 3 1 0 B 3 2 1 1 C 3 1 1 1 D 2 1 1	A W1 4 7 B W1 0 2 C W1 2 3 D W1 3 4 AVG_WAIT 1.75	PASS
3	**Resource pool R=5: big job blocks, no unfair overtaking** custom	2 3 5 0 R1 1 4 3 0 R2 1 2 3 1 R3 1 1 2	R1 W1 0 4 R2 W1 4 6 R3 W2 4 5 AVG_WAIT 2.33	PASS
4	**Boundary: equal priority and arrival -> input order** boundary	3 3 5 X 1 1 1 5 Y 1 1 1 5 Z 1 1 1	X W1 5 6 Y W2 5 6 Z W3 5 6 AVG_WAIT 0.00	PASS
5	**Invalid: resources above 100** invalid	1 1 0 J1 1 1 200	INPUT ERROR: job J1: value outside con straints	PASS
6	**Max size: w=64, n=200000** performance	random arrivals, priorities, durations , resources	finished in 3.97 s	PASS

Q10. Tkinter Assignment Tracker with File Persistence

CO-1, CO-4, CO-5 | Bloom L6 Create | Tkinter, event handling, persistence, validation, CSV/JSON

Problem

A Tkinter application to add students, add submissions, update marks, filter by pending/completed and export a CSV report (enrollment, name, assignment, status, marks, remarks). Data must persist locally and inputs must be validated.

Approach

- **Model / view separation:** `TrackerModel` holds the data, validation and file handling with no GUI code, so it is unit-tested without a display. `TrackerApp` only reads widgets and calls model methods.
- **Widgets (more than 5):** Entry, Text (remarks), Combobox (student), Checkbutton (submitted), Radiobutton (filter), Treeview with Scrollbar (table), Button, Menu (File → Export / Exit), LabelFrame, messagebox and filedialog.
- **Validation:** enrollment is 4 to 15 digits; names must start with a letter; marks must be numeric and between 0 and the maximum; the student must exist before a submission is added. Errors are shown in a message box and nothing is saved.
- **Persistence:** JSON is saved after every change and written atomically (temporary file, then `os.replace`), so a crash cannot leave a half-written data file. Records are keyed by (enrollment, assignment), so saving the same assignment again updates it.
- Status is derived from the data: with marks it is Completed, without marks it is Pending. Selecting a row fills the form for editing, and the search box filters as you type.

Complexity

Time O(r) to filter or export r records (plus O(r log r) to sort the visible rows). **Space** O(r). With 5,000 records a full table refresh measured about 50 ms, which shows no visible freeze.

> Requires Tkinter: it is included with the python.org installers for Windows and macOS; on Ubuntu or Debian install `python3-tk`.

How to run

```
python ENROLLMENT_1_Q10.py
```

Source code — ENROLLMENT_1_Q10.py

```python
"""
Q10. Tkinter Assignment Tracker with File Persistence
Course : Programming with Python (202044504) - Assignment 1
CO     : CO-1, CO-4, CO-5

Features
    * Add student (enrollment + name)
    * Add / update a submission (assignment, marks, max marks, remarks)
    * Update marks of the selected submission
    * Filter by All / Pending / Completed and search by enrollment or name
    * Export a CSV report: enrollment,name,assignment,status,marks,remarks
    * Data persists in tracker_data.json (saved after every change,
      written atomically: temp file + os.replace, so a crash never leaves
      a half-written file)

Widgets used (more than the required 5)
    Entry, Text, Combobox, Checkbutton, Radiobutton, Treeview, Button,
    Menu, Label, LabelFrame, Scrollbar, messagebox, filedialog

Structure
    TrackerModel  - data + validation + persistence, no GUI code
                    (can be unit-tested without a display)
    TrackerApp    - Tkinter GUI that only calls TrackerModel methods

Status rule: a submission with marks is "Completed"; without marks it is
"Pending" (tick "Submitted" to enter marks).

Run   : python ENROLLMENT_1_Q10.py         (needs Tkinter; Python 3.10+)
Time  : O(r) report / filter     Space : O(r)
"""

import csv
import json
import os
import re
import tempfile

ENROLL_RE = re.compile(r"^\d{4,15}$")
NAME_RE = re.compile(r"^[A-Za-z][A-Za-z .'-]{0,59}$")
DATA_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "tracker_data.json")
CSV_HEADER = ["enrollment", "name", "assignment", "status", "marks", "remarks"]


class ValidationError(Exception):
    """Raised when user input is not acceptable; message is shown to the user."""


def fmt_marks(value):
    """18.0 -> '18', 17.5 -> '17.5', None -> ''."""
    if value is None:
        return ""
    return str(int(value)) if float(value).is_integer() else str(value)


# ============================== MODEL ==================================
class TrackerModel:
    def __init__(self, path=DATA_FILE):
        self.path = path
        self.students = {}        # enrollment -> name
        self.submissions = {}     # (enrollment, assignment_lower) -> record dict
        self.load()

    # ---------- persistence ----------
    def load(self):
        if not os.path.exists(self.path):
            return
        try:
            with open(self.path, encoding="utf-8") as fh:
                data = json.load(fh)
            self.students = dict(data.get("students", {}))
            for rec in data.get("submissions", []):
                self.submissions[(rec["enrollment"], rec["assignment"].lower())] = rec
```

```
        except (OSError, ValueError, KeyError, TypeError) as err:
            raise ValidationError(f"Could not read {self.path}: {err}") from None

    def save(self):
        data = {"students": self.students, "submissions": list(self.submissions.values())}
        folder = os.path.dirname(self.path) or "."
        fd, tmp = tempfile.mkstemp(dir=folder, suffix=".tmp")
        with os.fdopen(fd, "w", encoding="utf-8") as fh:
            json.dump(data, fh, indent=1)
        os.replace(tmp, self.path)              # atomic on Windows and Linux

    # ---------- validation helpers ----------
    @staticmethod
    def _check_enrollment(enr):
        enr = enr.strip()
        if not ENROLL_RE.match(enr):
            raise ValidationError("Enrollment must be 4-15 digits.")
        return enr

    @staticmethod
    def _parse_marks(marks_text, max_text):
        try:
            max_marks = float(max_text)
        except ValueError:
            raise ValidationError("Max marks must be a number.") from None
        if not 0 < max_marks <= 1000:
            raise ValidationError("Max marks must be between 1 and 1000.")
        try:
            marks = float(marks_text)
        except ValueError:
            raise ValidationError("Marks must be a number.") from None
        if not 0 <= marks <= max_marks:
            raise ValidationError(f"Marks must be between 0 and {fmt_marks(max_marks)}.")
        return marks, max_marks

    # ---------- operations ----------
    def add_student(self, enrollment, name):
        enr = self._check_enrollment(enrollment)
        name = " ".join(name.split())
        if not NAME_RE.match(name):
            raise ValidationError("Name must start with a letter and use only letters, "
                                  "spaces, . ' - (max 60).")
        if enr in self.students:
            raise ValidationError(f"Student {enr} already exists.")
        self.students[enr] = name
        self.save()

    def add_submission(self, enrollment, assignment, submitted, marks_text="",
                       max_text="20", remarks=""):
        """Add a new submission or replace the existing one (same student + assignment)."""
        enr = self._check_enrollment(enrollment)
        if enr not in self.students:
            raise ValidationError(f"Student {enr} does not exist. Add the student first.")
        assignment = " ".join(assignment.split())
        if not 1 <= len(assignment) <= 60:
            raise ValidationError("Assignment name is required (max 60 characters).")
        remarks = " ".join(remarks.split())       # no newlines inside CSV cells
        if len(remarks) > 200:
            raise ValidationError("Remarks can be at most 200 characters.")
        if submitted:
            marks, max_marks = self._parse_marks(marks_text, max_text)
            status = "Completed"
        else:
            marks, max_marks, status = None, None, "Pending"
        self.submissions[(enr, assignment.lower())] = {
            "enrollment": enr, "assignment": assignment, "status": status,
            "marks": marks, "max_marks": max_marks, "remarks": remarks}
        self.save()

    def update_marks(self, enrollment, assignment, marks_text, max_text):
        key = (enrollment, assignment.lower())
        if key not in self.submissions:
```

```
            raise ValidationError("Select an existing submission first.")
        marks, max_marks = self._parse_marks(marks_text, max_text)
        rec = self.submissions[key]
        rec.update(marks=marks, max_marks=max_marks, status="Completed")
        self.save()

    def rows(self, status="All", text=""):
        """Filtered rows for display, sorted by enrollment then assignment. O(r log r)."""
        text = text.strip().lower()
        result = []
        for rec in self.submissions.values():
            if status != "All" and rec["status"] != status:
                continue
            name = self.students.get(rec["enrollment"], "")
            if text and text not in rec["enrollment"] and text not in name.lower():
                continue
            result.append((rec["enrollment"], name, rec["assignment"], rec["status"],
                           fmt_marks(rec["marks"]), rec["remarks"]))
        result.sort(key=lambda r: (r[0], r[2].lower()))
        return result

    def export_csv(self, path, status="All", text=""):
        with open(path, "w", newline="", encoding="utf-8") as fh:
            writer = csv.writer(fh)
            writer.writerow(CSV_HEADER)
            rows = self.rows(status, text)
            writer.writerows(rows)
        return len(rows)


# ============================== GUI ===================================
def run_gui(model=None):
    import tkinter as tk
    from tkinter import filedialog, messagebox, ttk

    class TrackerApp:
        def __init__(self, root, model):
            self.root = root
            self.model = model
            root.title("Assignment Tracker")
            root.geometry("980x620")
            root.minsize(860, 560)
            self._build_menu()
            self._build_forms()
            self._build_table()
            self.refresh()

        # ---------- layout ----------
        def _build_menu(self):
            menubar = tk.Menu(self.root)
            file_menu = tk.Menu(menubar, tearoff=False)
            file_menu.add_command(label="Export CSV...", command=self.export)
            file_menu.add_separator()
            file_menu.add_command(label="Exit", command=self.root.destroy)
            menubar.add_cascade(label="File", menu=file_menu)
            self.root.config(menu=menubar)

        def _build_forms(self):
            top = ttk.Frame(self.root, padding=8)
            top.pack(fill="x")

            stu = ttk.LabelFrame(top, text="Add student", padding=8)
            stu.pack(side="left", fill="y", padx=(0, 8))
            self.enr_var, self.name_var = tk.StringVar(), tk.StringVar()
            ttk.Label(stu, text="Enrollment").grid(row=0, column=0, sticky="w")
            ttk.Entry(stu, textvariable=self.enr_var, width=18).grid(row=0, column=1, pady=2)
            ttk.Label(stu, text="Name").grid(row=1, column=0, sticky="w")
            ttk.Entry(stu, textvariable=self.name_var, width=18).grid(row=1, column=1, pady=2)
            ttk.Button(stu, text="Add student", command=self.add_student).grid(
                row=2, column=0, columnspan=2, sticky="ew", pady=(6, 0))

            sub = ttk.LabelFrame(top, text="Submission", padding=8)
```

```
            sub.pack(side="left", fill="both", expand=True)
            self.student_var = tk.StringVar()
            self.assign_var = tk.StringVar(value="Assignment 1")
            self.marks_var, self.max_var = tk.StringVar(), tk.StringVar(value="20")
            self.submitted_var = tk.BooleanVar(value=True)

            ttk.Label(sub, text="Student").grid(row=0, column=0, sticky="w")
            self.student_box = ttk.Combobox(sub, textvariable=self.student_var,
                                            state="readonly", width=26)
            self.student_box.grid(row=0, column=1, columnspan=3, sticky="w", pady=2)
            ttk.Label(sub, text="Assignment").grid(row=1, column=0, sticky="w")
            ttk.Entry(sub, textvariable=self.assign_var, width=28).grid(
                row=1, column=1, columnspan=3, sticky="w", pady=2)
            ttk.Checkbutton(sub, text="Submitted", variable=self.submitted_var,
                            command=self._toggle_marks).grid(row=2, column=0, sticky="w")
            ttk.Label(sub, text="Marks").grid(row=2, column=1, sticky="e")
            self.marks_entry = ttk.Entry(sub, textvariable=self.marks_var, width=6)
            self.marks_entry.grid(row=2, column=2, sticky="w", padx=4)
            ttk.Label(sub, text="/").grid(row=2, column=3, sticky="w")
            self.max_entry = ttk.Entry(sub, textvariable=self.max_var, width=6)
            self.max_entry.grid(row=2, column=3, sticky="w", padx=(14, 0))
            ttk.Label(sub, text="Remarks").grid(row=0, column=4, sticky="nw", padx=(12, 4))
            self.remarks = tk.Text(sub, width=30, height=4, wrap="word")
            self.remarks.grid(row=0, column=5, rowspan=3, sticky="nsew")
            btns = ttk.Frame(sub)
            btns.grid(row=3, column=0, columnspan=6, sticky="w", pady=(8, 0))
            ttk.Button(btns, text="Save submission", command=self.add_submission).pack(side="left")
            ttk.Button(btns, text="Update marks", command=self.update_marks).pack(side="left", padx=6)
            ttk.Button(btns, text="Export CSV", command=self.export).pack(side="left")
            sub.columnconfigure(5, weight=1)

        def _build_table(self):
            bar = ttk.Frame(self.root, padding=(8, 0))
            bar.pack(fill="x")
            self.filter_var = tk.StringVar(value="All")
            ttk.Label(bar, text="Show:").pack(side="left")
            for value in ("All", "Pending", "Completed"):
                ttk.Radiobutton(bar, text=value, value=value, variable=self.filter_var,
                                command=self.refresh).pack(side="left", padx=4)
            self.search_var = tk.StringVar()
            self.search_var.trace_add("write", lambda *_: self.refresh())
            ttk.Entry(bar, textvariable=self.search_var, width=24).pack(side="right")
            ttk.Label(bar, text="Search:").pack(side="right", padx=4)

            frame = ttk.Frame(self.root, padding=8)
            frame.pack(fill="both", expand=True)
            cols = ("enrollment", "name", "assignment", "status", "marks", "remarks")
            widths = (100, 160, 140, 90, 60, 300)
            self.tree = ttk.Treeview(frame, columns=cols, show="headings", selectmode="browse")
            for col, width in zip(cols, widths):
                self.tree.heading(col, text=col.title())
                self.tree.column(col, width=width, anchor="w")
            scroll = ttk.Scrollbar(frame, orient="vertical", command=self.tree.yview)
            self.tree.configure(yscrollcommand=scroll.set)
            self.tree.pack(side="left", fill="both", expand=True)
            scroll.pack(side="right", fill="y")
            self.tree.bind("<<TreeviewSelect>>", self.on_select)

            self.status_var = tk.StringVar()
            ttk.Label(self.root, textvariable=self.status_var, padding=(8, 2)).pack(fill="x")

        # ---------- helpers ----------
        def _toggle_marks(self):
            state = "normal" if self.submitted_var.get() else "disabled"
            self.marks_entry.config(state=state)
            self.max_entry.config(state=state)

        def _selected_enrollment(self):
            return self.student_var.get().split(" - ")[0]

        def _guard(self, action, success_msg):
            try:
```

```
                action()
            except ValidationError as err:
                messagebox.showerror("Invalid input", str(err), parent=self.root)
                return False
            except OSError as err:
                messagebox.showerror("File error", str(err), parent=self.root)
                return False
            self.refresh()
            self.status_var.set(success_msg)
            return True

        def refresh(self):
            """Rebuild the table for the current filter (fast enough for 5000+ rows)."""
            rows = self.model.rows(self.filter_var.get(), self.search_var.get())
            self.tree.delete(*self.tree.get_children())
            for row in rows:
                self.tree.insert("", "end", values=row)
            names = [f"{e} - {n}" for e, n in sorted(self.model.students.items())]
            self.student_box["values"] = names
            if names and not self.student_var.get():
                self.student_var.set(names[0])
            self.status_var.set(f"{len(rows)} record(s) shown, "
                                f"{len(self.model.submissions)} total")

        # ---------- event handlers ----------
        def add_student(self):
            enr = self.enr_var.get().strip()
            if self._guard(lambda: self.model.add_student(enr, self.name_var.get()),
                           f"Student {enr} added."):
                self.student_var.set(f"{enr} - {self.model.students[enr]}")
                self.enr_var.set("")
                self.name_var.set("")

        def add_submission(self):
            self._guard(lambda: self.model.add_submission(
                self._selected_enrollment(), self.assign_var.get(), self.submitted_var.get(),
                self.marks_var.get(), self.max_var.get(), self.remarks.get("1.0", "end")),
                "Submission saved.")

        def update_marks(self):
            self._guard(lambda: self.model.update_marks(
                self._selected_enrollment(), self.assign_var.get(),
                self.marks_var.get(), self.max_var.get()), "Marks updated.")

        def on_select(self, _event):
            sel = self.tree.selection()
            if not sel:
                return
            enr, name, assignment, status, marks, remarks = self.tree.item(sel[0], "values")
            self.student_var.set(f"{enr} - {name}")
            self.assign_var.set(assignment)
            self.submitted_var.set(status == "Completed")
            self._toggle_marks()
            self.marks_var.set(marks)
            rec = self.model.submissions.get((enr, assignment.lower()), {})
            self.max_var.set(fmt_marks(rec.get("max_marks")) or "20")
            self.remarks.delete("1.0", "end")
            self.remarks.insert("1.0", remarks)

        def export(self):
            path = filedialog.asksaveasfilename(
                parent=self.root, defaultextension=".csv", initialfile="assignment_report.csv",
                filetypes=[("CSV files", "*.csv")])
            if path:
                count = []
                self._guard(lambda: count.append(self.model.export_csv(
                    path, self.filter_var.get(), self.search_var.get())), "Exported.")
                if count:
                    self.status_var.set(f"Exported {count[0]} row(s) to {path}")

    root = tk.Tk()
    try:
```

```
        model = model or TrackerModel()
    except ValidationError as err:
        messagebox.showerror("Data file error", str(err))
        root.destroy()
        return None
    app = TrackerApp(root, model)
    return root, app


if __name__ == "__main__":
    created = run_gui()
    if created:
        created[0].mainloop()
```

Test cases — 6 of 6 passed

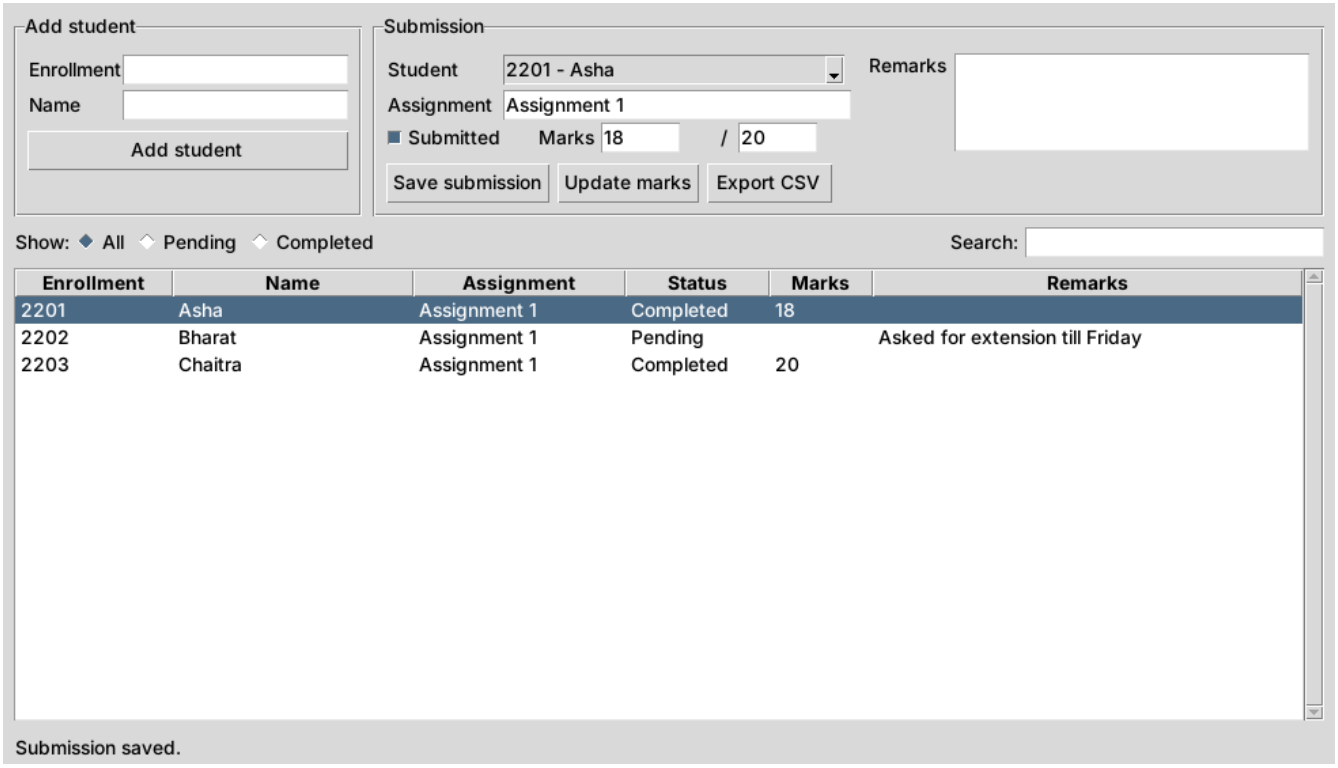
#	Test case	Input	Output (expected = actual)	Result
1	**Sample: add 2201 Asha, Assignment 1 = 18/20, export** sample	`Add student 2201, Asha; Add Assignment 1 marks 18/20; Export CSV`	`enrollment,name,assignment,status,marks,remarks` `2201,Asha,Assignment 1,Completed,18,`	**PASS**

2	**Validation rejects bad input**
	invalid
		`bad enrollment 'A22'`
		`duplicate student`
		`marks 21 out of 20`
		`marks 'abc'`
		`unknown student`
		`empty assignment`
			`Enrollment must be 4-15 digits.`
			`Student 2201 already exists.`
			`Marks must be between 0 and 20.`
			`Marks must be a number.`
			`Student 9999 does not exist. Add the student first.`
			`Assignment name is required (max 60 characters).`
				PASS

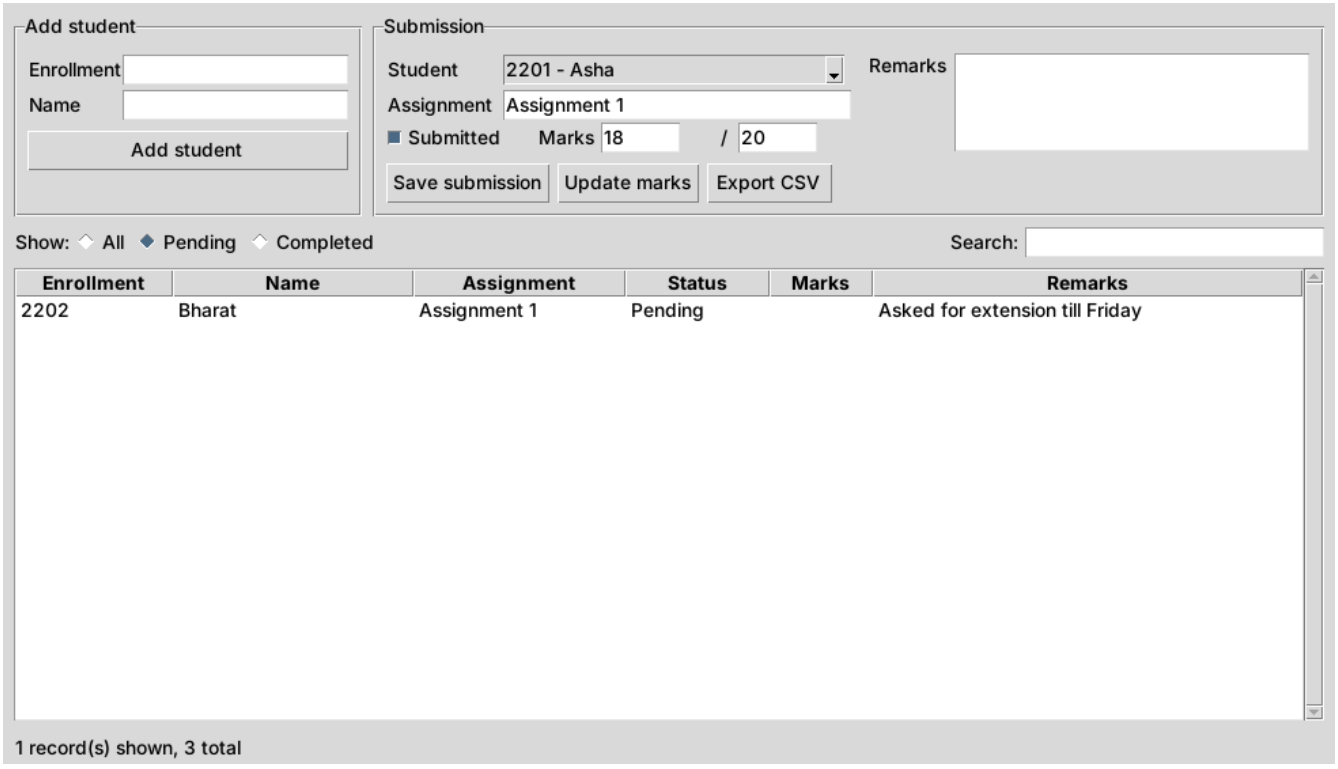
3	**Update marks (0 and full marks), persistence after reload, filter**
	boundary
		`2202 pending -> update to 20/20; 2201 A2 = 0/20; reload JSON; filter Completed`	`2201,Asha,Assignment 1,Completed,18,`
			`2201,Asha,Assignment 2,Completed,0,late, 0 marks`
			`2202,Bharat,Assignment 1,Completed,20, asked for extension`
				PASS

4	**Pending filter and name search** custom	`filter Pending; search 'bha'`	`2201,Asha,Assignment 3,Pending,, | search 'bha': 1`	**PASS**
5	**5000 records: average filter time (model)** performance	`5000 submissions, filters All / Pending / Completed`	`2.5 ms`	**PASS**
6	**GUI: refresh real Treeview with 5000 rows** performance	`5000 submissions shown in the Tkinter table, filters All / Pending / Completed`	`All 51.2 ms, Pending 24.1 ms, Completed 37.7 ms`	**PASS**

Screenshots



Main window after adding three students and their Assignment 1 submissions (2201 selected for editing)



Filter set to Pending: only the submission without marks is shown

Exported CSV report (from the session above)

```
enrollment,name,assignment,status,marks,remarks
2201,Asha,Assignment 1,Completed,18,
2202,Bharat,Assignment 1,Pending,,Asked for extension till Friday
2203,Chaitra,Assignment 1,Completed,20,
```